PES UNIVERSITY · DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Database Management Systems · Experiential Learning – Level 3 Project · Section D

TalentMatch

AI-Assisted Resume-to-Job Matching and Recruitment Management System

Project report following the 14 documentation items of §14 of the course requirements. Database option: **SQL + Vector Database** (§3, Option 2): PostgreSQL 17 with the pgvector 0.8.7 extension, behind a Next.js web application.

Team member	SRN
Rohith Karning	PES1UG24AM229
Saanvi Maharana	PES1UG24AM237

Repository: github.com/rohithkarning-debug/TalentMatch
Content source: the repository's `docs/` folder (project report, data dictionary, ER diagram, screenshots) and the course requirements handout. Figures are from the final run on 16 Oct 2026 (database rebuilt from scratch with migrations 001–024).

Contents

1 Problem statement
2 Objectives
3 Scope
4 Functional requirements
5 ER diagram
6 Relational schema
7 Database choice and justification
8 Data dictionary
9 SQL and vector database implementation
10 Important queries
11 System architecture
12 Application screenshots
13 Testing results
14 Conclusion and future enhancements
15 Team contribution
A Requirements compliance matrix
B Full data dictionary

Project at a glance

16 tables (public schema)	**22** foreign keys	**57** CHECK constraints	**46** indexes (2 HNSW, 3 partial)
19 triggers (1 deferred)	**6** stored functions	**3** views	**8** multi-statement transactions
24 migrations (001–024)	**41** documented queries	**85** unit tests passing	**147** integration tests passing

Lab evaluation checklist (project status evaluation, 26 Oct)

Component checked	Where it is met in TalentMatch	Status
Minimum 5 tables in the ER diagram	16 tables with 22 foreign keys, including specialisation (users → recruiters / candidates) and three M:N junction tables. Section 5, Section 6, Appendix B.	Met
RDBMS + second database paradigm	PostgreSQL relational schema + pgvector vector store (`vector(384)`, HNSW, cosine similarity) in the same database. The handout's §3 lists this as Option 2, "PostgreSQL with pgvector". Section 7, Section 9.6.	Met
Reports in the project demo	Admin dashboard built on three views and `generate_hiring_pipeline_report()`: funnel per stage, top skills, score by outcome, per-company and per-job tables (screenshot 25). Report queries Q3, Q10, Q11, Q16, Q37. Sections 9.3, 9.4, 10, 12.	Met
Novelty in the project	Candidates ranked by meaning (real cosine score stored on each application and recomputable in SQL), shown with required-skill coverage; skill-gap view with `EXCEPT`; the hiring workflow enforced inside the database (status state machine, deferred constraint trigger, cross-table offer/interview rules). Sections 1, 9.2, 9.6.	Met

1 Problem statement

Recruiters at small and mid-sized companies receive many resumes for every opening. They usually shortlist by hand or with keyword search. This is slow and inconsistent, and it misses good candidates whose resumes describe the same experience in different words. A resume that says *"trained transformer text classifiers"* fits a job asking for *"NLP pipelines"*, yet the two share almost no keywords.

Most simple applicant-tracking tools only store candidate and job records (CRUD). They do not understand the meaning of a resume, and they do not enforce the hiring workflow: an application can jump from "applied" straight to "offered", an offer can exist for a rejected candidate, and counters and histories drift out of sync.

TalentMatch addresses both problems:

- it **ranks candidates by meaning**, using sentence embeddings stored in PostgreSQL with pgvector and compared with cosine similarity;
- it **manages the whole hiring pipeline** (apply → shortlist → interview → offer → accept/decline, or reject) with the rules enforced **inside the database** through constraints, triggers, functions and transactions, so the data stays consistent no matter which program changes it.

2 Objectives

1. Design and implement a normalised relational database for candidates, recruiters, companies, jobs, skills, resumes, applications, interviews, offers and notifications.
2. Implement vector-based semantic matching of resumes to jobs with embeddings and cosine similarity, going beyond keyword search.
3. Build a role-based Next.js web application (admin, recruiter, candidate) that talks to the database only through a server-side API layer.
4. Demonstrate advanced database concepts: transactions, triggers (including a deferred constraint trigger), stored functions, views, indexing (B-tree, partial, HNSW) and vector similarity search.
5. Enforce data validation and integrity at the database level (CHECK, NOT NULL, UNIQUE, FK, triggers), not only in the application.
6. Follow basic security practice: bcrypt password hashes, signed httpOnly session cookies, server-side role checks, parameterized SQL, no secrets in code.
7. Prove every claim with automated tests and real outputs.

3 Scope

In scope

- Candidate self-registration, profile, skills with proficiency, resume upload (PDF or TXT, at most 5 MB, at least 50 characters of text).
- Recruiters (created by the admin) post and manage jobs for their own company, see applicants ranked by semantic match score, search all candidates, change application status, schedule interviews and make offers.
- Candidates browse and filter open jobs, see their skill gap for a job, apply, track applications, see interviews and accept or decline offers.

- In-app notifications for every pipeline event.
- An admin dashboard with hiring-pipeline analytics, and admin management of companies and recruiter accounts.
- Local embedding model (no external AI service), PostgreSQL 17 + pgvector in one Docker container.

Out of scope (listed under future enhancements): email/SMS notifications, external job boards, payroll/onboarding, password reset and email verification, multi-server deployment, languages other than English for matching.

4 Functional requirements

ID	Role	Requirement	Where it is implemented
FR-01	Anyone	Register as a candidate (never as admin/recruiter)	`POST /api/auth/register`, users + candidates in one transaction
FR-02	All	Log in / log out; session in an httpOnly signed cookie; 5 failed attempts per 15 min → 429	`/api/auth/*`
FR-03	Candidate	Edit profile; add/remove skills with proficiency 1–5	`/api/candidate/profile`, `/api/candidate/skills`
FR-04	Candidate	Upload, list, delete resumes; choose a primary resume; resume is embedded after upload	`/api/candidate/resumes`
FR-05	Candidate	Browse open jobs with keyword search and filters (location, work mode, type, experience)	`/api/candidate/jobs`
FR-06	Candidate	See the skill gap for a job (have / below level / missing)	`/api/candidate/jobs/:id` (`EXCEPT` query)
FR-07	Candidate	Apply with a chosen resume; get a real cosine match score; one application per job	`POST /api/candidate/applications` (transaction)
FR-08	Candidate	Track applications with full status history; see interviews; accept/decline offers before expiry	`/api/candidate/applications`, `/interviews`, `/offers`
FR-09	Recruiter	Create, edit, open/close, delete jobs of their own company, with required skills	`/api/recruiter/jobs`
FR-10	Recruiter	See applicants ranked by match score with matched/required skills; search all candidates by semantic similarity	`/api/recruiter/jobs/:id/applications`, `/matches`
FR-11	Recruiter	Move applications through the allowed stages; invalid moves are refused with the database's message	`PATCH /api/recruiter/applications/:id`
FR-12	Recruiter	Schedule, reschedule, cancel and complete interviews (rating 1–5, feedback)	`/api/recruiter/applications/:id/interviews`, `/api/recruiter/interviews/:id`
FR-13	Recruiter	Make and withdraw offers	`/api/recruiter/applications/:id/offer`, `/api/recruiter/offers/:id/withdraw`
FR-14	All	Notifications inbox; mark one or all as read	`/api/notifications`

ID	Role	Requirement	Where it is implemented
FR-15	Admin	Dashboard: KPIs, funnel per stage, top skills, score by outcome, per-company and per-job tables	`GET /api/admin/dashboard`
FR-16	Admin	Manage companies (delete refused while in use) and recruiter accounts (create, edit, deactivate)	`/api/admin/companies`, `/api/admin/recruiters`

Non-functional: every route checks the role on the server; ownership is enforced in SQL; credentials only in environment variables; parameterized SQL only; database-level validation; the app runs from a clean clone using the README.

5 ER diagram

The full diagram with every attribute is in er-diagram.md (Mermaid source) and as an image:

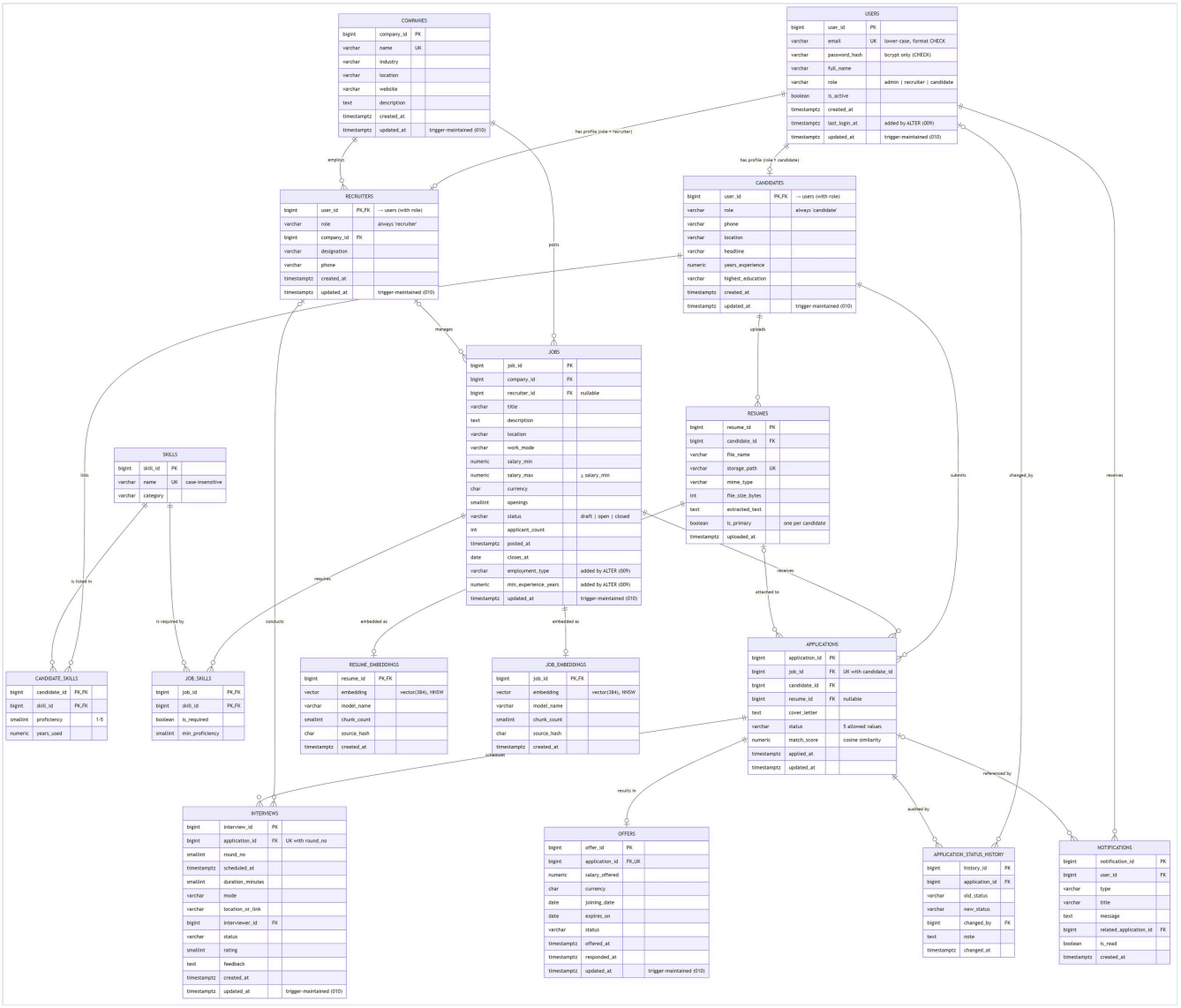
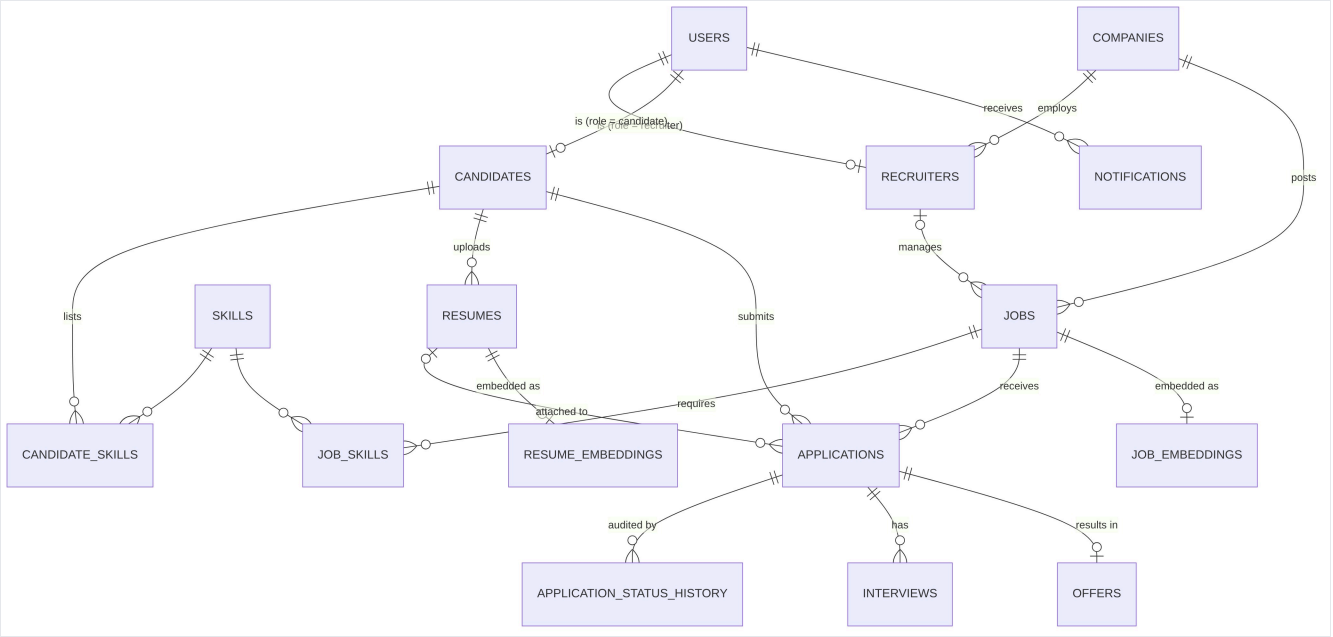


Figure: full ER diagram with every attribute (`docs/er-diagram.png`, generated from `docs/er-diagram.md`). Zoom in for column detail.

Relationship overview (cardinalities):



Relationship overview (cardinalities), rendered from the Mermaid source in `docs/project-report.md`.

Key design points:

- **Specialisation:** `users` is the login identity; `recruiters` and `candidates` are 1 : 0..1 profiles whose PK is also the FK. A composite FK `(user_id, role)` → `users(user_id, role)` guarantees that only a user with role `recruiter` can have a recruiter profile (and likewise for candidates).
- **M:N relationships** are junction tables: `candidate_skills`, `job_skills`, and `applications` (candidates ↔ jobs, with its own attributes).
- **Weak-like entities:** interviews (`UNIQUE (application_id, round_no)`), offers (`UNIQUE (application_id)`, at most one per application), embeddings (PK = FK, at most one per resume/job).

6 Relational schema

16 tables in schema `public` (migration bookkeeping is kept separately in `tm_meta.schema_migrations`). underline = primary key, *italic* = foreign key, UK = unique. Full version with every FK delete rule: relational-schema.md.

```
users(user_id, email UK, password_hash, full_name, role, is_active, created_at, last_login_at,
updated_at)
      UK (user_id, role)
companies(company_id, name UK, industry, location, website, description, created_at, updated_at)
recruiters(user_id → users, role, company_id → companies, designation, phone, created_at, updated_at)
      FK (user_id, role) → users(user_id, role);  UK (user_id, company_id)
candidates(user_id → users, role, phone, location, headline, years_experience, highest_education,
created_at, updated_at)
      FK (user_id, role) → users(user_id, role)
skills(skill_id, name UK (case-insensitive), category)
candidate_skills(candidate_id → candidates, skill_id → skills, proficiency, years_used)      PK
(candidate_id, skill_id)
jobs(job_id, company_id → companies, recruiter_id, title, description, location, work_mode, salary_min,
salary_max,
     currency, openings, status, applicant_count, posted_at, closes_at, employment_type,
min_experience_years, updated_at)
      FK (recruiter_id, company_id) → recruiters(user_id, company_id)
job_skills(job_id → jobs, skill_id → skills, is_required, min_proficiency)                    PK
(job_id, skill_id)
resumes(resume_id, candidate_id → candidates, file_name, storage_path UK, mime_type, file_size_bytes,
        extracted_text, is_primary, uploaded_at)                 partial UK (candidate_id) WHERE
is_primary
applications(application_id, job_id → jobs, candidate_id → candidates, resume_id → resumes,
cover_letter,
             status, match_score, applied_at, updated_at)         UK (job_id, candidate_id)
application_status_history(history_id, application_id → applications, old_status, new_status,
                           changed_by → users, note, changed_at)
interviews(interview_id, application_id → applications, round_no, scheduled_at, duration_minutes, mode,
           location_or_link, interviewer_id → recruiters, status, rating, feedback, created_at,
updated_at)
           UK (application_id, round_no)
offers(offer_id, application_id → applications UK, salary_offered, currency, joining_date, expires_on,
       status, offered_at, responded_at, updated_at)
notifications(notification_id, user_id → users, type, title, message, related_application_id →
applications,
              is_read, created_at)
resume_embeddings(resume_id → resumes (PK), embedding vector(384), model_name, chunk_count,
source_hash, created_at)
job_embeddings(job_id → jobs (PK), embedding vector(384), model_name, chunk_count, source_hash,
created_at)
```

Constraint totals in the live database: 16 primary keys, 22 foreign keys, 8 UNIQUE constraints, 57 CHECK constraints, 1 constraint trigger, 46 indexes (2 HNSW, 3 partial).

ON DELETE rules were chosen per relationship: CASCADE where the child cannot exist without the parent (profile, skills list, resumes, embeddings, history, interviews, offers), RESTRICT where deleting would destroy business history (company with recruiters or jobs, job with applications, skill in use), SET NULL where the child is still valuable without the link (audit row whose author was deleted, job whose recruiter left).

Normalisation (1NF–3NF):

- **1NF** — every value is atomic. A candidate's skills are rows in `candidate_skills`, not a comma-separated text column.
- **2NF** — in the composite-key tables `candidate_skills (candidate_id, skill_id)` and `job_skills (job_id, skill_id)`, every non-key column (`proficiency`, `years_used`, `is_required`, `min_proficiency`) depends on the *whole* key, not on one part. The skill's name and category depend only on `skill_id`, so they live in `skills`.
- **3NF** — no non-key column depends on another non-key column. A job stores `company_id`, not the company name or website; those depend on the company and live in `companies`.
- **Deliberate denormalisation:** `jobs.applicant_count` repeats `COUNT(applications)` so job lists are fast. A trigger keeps it exact (tested: 0 mismatches after seed and after every test run).

7 Database choice and justification

TalentMatch uses **Option 2 of the requirements: SQL + vector database**, both inside **one PostgreSQL 17 database with the pgvector 0.8.7 extension**.

Data	Nature	Stored as	Why this paradigm
Users, companies, jobs, applications, interviews, offers, skills, notifications	Highly structured, linked by relationships, must stay consistent (one application per job per candidate, one offer per application, valid status changes)	Relational tables	PK/FK, UNIQUE, CHECK, triggers and ACID transactions guarantee integrity; joins and aggregates serve the analytics
Resume text and job descriptions	Unstructured free text where **meaning** matters more than exact words	`vector(384)` columns in `resume_embeddings` and `job_embeddings`	Embeddings put texts of similar meaning close together; cosine similarity ranks candidates by relevance, which SQL `LIKE` or full-text search cannot do

Why pgvector in the same database instead of a separate vector store (Pinecone, Qdrant, Chroma)?

1. **One query mixes both kinds of data.** Ranking candidates is a vector sort (`ORDER BY embedding <=> job_vector`) filtered by relational columns (primary resume, active user, applied to this job) and joined with skill counts — one SQL function, no syncing of IDs between two systems.
2. **Referential integrity for vectors.** Each embedding row's PK is a real FK with `ON DELETE CASCADE`, so deleting a resume deletes its vector and a vector can never point at a missing resume.
3. **Database-level validation of vectors.** The column type enforces 384 dimensions and a CHECK refuses vectors that are not unit length.
4. **Same transactions, security and backups**: one container, one set of credentials, one role-checked API.
5. **The scale fits.** A recruitment system holds thousands of resumes, not billions; pgvector with an HNSW index handles millions.

The deciding factor is the nature of the data: every vector **belongs to** exactly one resume or job row, so it should live under that row's foreign key.

8 Data dictionary

Column-level detail (type, NULL, default, every CHECK) for all 16 tables is in data-dictionary.md. Summary:

Table	Purpose	Key columns and rules
`users`	Login identity	`email` UNIQUE, lower-case CHECK; `password_hash` CHECK bcrypt format (plaintext rejected); `role` IN (admin, recruiter, candidate); `is_active` for soft delete
`companies`	Hiring organisations	`name` UNIQUE; `website` must start with http(s)
`recruiters`	Recruiter profile	PK = FK `(user_id, role)`; `company_id` FK RESTRICT
`candidates`	Candidate profile	PK = FK `(user_id, role)`; `years_experience` 0–60
`skills`	Skill catalogue	unique `lower(name)`; `category` from 8 values
`candidate_skills`	Candidate ↔ skill	composite PK; `proficiency` 1–5
`jobs`	Job postings	`status` draft/open/closed; `salary_max ≥ salary_min`; `description` ≥ 50 chars; `applicant_count` trigger-maintained
`job_skills`	Job ↔ skill	composite PK; `is_required`; `min_proficiency` 1–5
`resumes`	Uploaded resumes	`mime_type` PDF/TXT only; size ≤ 5 MB; text ≥ 50 chars; one primary per candidate (partial unique index)
`applications`	Centre of the workflow	UNIQUE (job, candidate); `status` 5 values guarded by triggers; `match_score` NUMERIC(5,4) between −1 and 1, real cosine similarity or NULL
`application_status_history`	Audit trail	written only by trigger; `old_status IS DISTINCT FROM new_status`
`interviews`	Interview rounds	UNIQUE (application, round); mode video/phone/onsite; link required unless phone; `rating` 1–5 only when completed
`offers`	Job offers	one per application; `salary_offered > 0`; `expires_on ≤ joining_date`; `responded_at` set exactly when answered
`notifications`	In-app inbox	`type` from 8 values; `title` not blank; partial index on unread
`resume_embeddings` / `job_embeddings`	Vector store	PK = FK CASCADE; `vector(384)` unit length; `model_name`; `source_hash` SHA-256 of the embedded text; HNSW index

9 SQL and vector database implementation

All schema changes are numbered migration files `db/migrations/001–024`, applied once each by `npm run db:migrate`, each in its own transaction, with a SHA-256 checksum recorded so an edited migration is refused.

Migrations	Content
001–008	Extensions, all 16 tables with PK/FK/UNIQUE/CHECK, embedding tables, B-tree / partial / HNSW indexes
009	`ALTER TABLE` demonstrations (add columns with defaults, add named CHECKs, change a column type, drop/re-add a constraint)
010–011	`updated_at` triggers; applicant count, initial status, status-transition guard, status history triggers
012–013	`update_application_status()`, `generate_hiring_pipeline_report()`; three views
014, 017	`updated_at` ignores `last_login_at` and `match_score` refreshes
015–016	`rank_candidates_for_job()` and `compute_match_score()` (pgvector)
018–019	Interview and offer guards, deferred constraint trigger, final rows frozen
020–022	Rejecting withdraws a pending offer; no completing an interview before its time; no rejecting after an accepted offer
023	No new offer while an interview round is still scheduled
024	Dates in rule messages shown in the app's display format (`16 Oct 2026`)

DDL coverage: `CREATE TABLE / INDEX / VIEW / FUNCTION / TRIGGER / EXTENSION / SCHEMA`; `ALTER TABLE … ADD COLUMN / ADD CONSTRAINT / ALTER COLUMN TYPE / DROP CONSTRAINT` (009, 018); `DROP` in `db/reset.sql` (`DROP VIEW`, `DROP FUNCTION`, `DROP TABLE … CASCADE`, `DROP SCHEMA`). **DML coverage:** INSERT (seed, apply, offers), UPDATE (status, profile, mark read), DELETE (resumes, skills, jobs, companies), SELECT with JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries, window functions, `EXCEPT`, CTEs.

9.1 Constraints (database-level validation)

```
-- 002: a plaintext password can never be stored, even by a bug
CONSTRAINT users_password_hash_bcrypt_check
    CHECK (password_hash ~ '^\$2[aby]\$\d{2}\$[./A-Za-z0-9]{53}$')
-- 002: a profile row can only belong to a user with the matching role
CONSTRAINT candidates_user_fkey FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role) ON DELETE CASCADE
-- 005: one application per job per candidate
CONSTRAINT applications_job_id_candidate_id_key UNIQUE (job_id, candidate_id)
```

9.2 Triggers (19 in total, 12 trigger functions)

Trigger	Table	Timing	Rule
`trg_*_set_updated_at` (8)	users, companies, recruiters, candidates, jobs, applications, interviews, offers	BEFORE UPDATE	`updated_at = now()` only if a real column changed
`trg_applications_applicant_count`	applications	AFTER INSERT/DELETE/UPDATE OF job_id	keeps `jobs.applicant_count` exact
`trg_applications_truncate_count`	applications	AFTER TRUNCATE	resets all counts to 0
`trg_applications_initial_status`	applications	BEFORE INSERT	new applications start as `applied`
`trg_applications_status_guard`	applications	BEFORE UPDATE OF status	only allowed transitions
`trg_applications_status_history`	applications	AFTER INSERT/UPDATE OF status	writes the audit row (who, why)
`trg_applications_stage_records`	applications	**constraint trigger, DEFERRABLE INITIALLY DEFERRED**	at COMMIT, `interviewed` needs an interview and `offered` a pending offer
`trg_applications_withdraw_offer`	applications	AFTER UPDATE OF status	rejecting withdraws a pending offer and notifies
`trg_applications_accepted_offer_guard`	applications	BEFORE UPDATE OF status	an accepted offer blocks rejection
`trg_interviews_guard`	interviews	BEFORE INSERT/UPDATE	future time; right stage; final rows frozen; complete only after the time
`trg_offers_guard`	offers	BEFORE INSERT/UPDATE	starts pending; dates valid; only pending rows change; no answer after expiry
`trg_offers_interview_guard`	offers	BEFORE INSERT	no offer while a round is still scheduled

Allowed status transitions (single source of truth: `is_valid_status_transition()`):

```
applied ──► shortlisted ──► interviewed ──► offered
   └──────────┴──────────────┴──────────────┴──► rejected (final)
```

Excerpt — the deferred check (migration 018):

```
CREATE CONSTRAINT TRIGGER trg_applications_stage_records
    AFTER UPDATE OF status ON applications
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW
    WHEN (NEW.status IN ('interviewed', 'offered') AND NEW.status IS DISTINCT FROM OLD.status)
    EXECUTE FUNCTION check_application_stage_records();
```

It is deferred because, inside the make-offer transaction, the status becomes `offered` *before* the offer row is inserted; only the state at COMMIT has to be consistent.

9.3 Stored functions

Function	Returns	Purpose
`update_application_status(app, status, actor, note)`	`applications` row	Locks the row (`FOR UPDATE`), checks the actor may change it (admin, or recruiter of the job's company), updates (triggers validate and log), notifies the candidate. Atomic
`generate_hiring_pipeline_report(company?, from?, to?)`	table, one row per job	Counts per stage, shortlist/offer rates, average days to shortlist, interviews done, offers accepted. Validates its parameters
`rank_candidates_for_job(job, limit, applicants_only)`	table	Semantic ranking with pgvector plus required-skill counts
`compute_match_score(resume, job)`	numeric	`round(1 - (resume <=> job), 4)` or NULL if either side is not embedded
`is_valid_status_transition(old, new)`	boolean	The transition table used by the guard and the tests
`local_today()`	date	Today's date in Asia/Kolkata (the server runs in UTC)

9.4 Views

View	One row per	Highlights
`recruiter_pipeline_view`	job	counts per stage, active in pipeline, average score, upcoming interviews (correlated subquery)
`candidate_application_view`	application	job, company, status, last status change and next interview (LATERAL), offer details
`company_hiring_summary`	company with applications	`GROUP BY` + `HAVING count(a.application_id) > 0`, rejection rate, offers made/accepted

9.5 Transactions

Transaction	Steps in one `BEGIN … COMMIT`	Code
Register	users + candidates	`src/app/api/auth/register/route.js`

Transaction	Steps in one `BEGIN … COMMIT`	Code
Apply to job	lock job `FOR SHARE`, lock resume `FOR SHARE`, INSERT application (3 triggers fire), notify recruiter	`src/lib/applications.js`
Schedule interview	lock application `FOR UPDATE`, next round number, INSERT interview, first round moves status to `interviewed`, notify	`src/lib/interviews.js`
Make offer	lock application, status → `offered`, INSERT offer, notify; deferred check at COMMIT	`src/lib/offers.js`
Accept / decline / withdraw offer	lock offer `FOR UPDATE OF o`, update, notify (withdraw also rejects the application)	`src/lib/offers.js`
Upload resume	lock candidate `FOR NO KEY UPDATE`, INSERT resume, maybe make it primary	`src/lib/candidate/resumes.js`
Create recruiter (admin)	users + recruiters	`src/lib/admin/recruiters.js`
Job create/edit/delete	jobs + job_skills, row locked	`src/app/api/recruiter/jobs`

Excerpt — apply to job (`src/lib/applications.js`):

```
await client.query("BEGIN");
const job = await client.query(
  "SELECT job_id, title, status, company_id, recruiter_id FROM jobs WHERE job_id = $1 FOR SHARE",
[jobId]);
...
const inserted = await client.query(
  `INSERT INTO applications (job_id, candidate_id, resume_id, cover_letter, match_score)
   VALUES ($1, $2, $3, $4, compute_match_score($3, $1))
   RETURNING ...`, [jobId, candidateId, resumeId, coverLetter]);
// notification INSERT ..., then COMMIT; any error -> ROLLBACK
```

`FOR SHARE` lets many candidates apply at the same time but makes a recruiter's "close job" wait until the application is committed. A duplicate application is not pre-checked with a SELECT (two requests could race); the UNIQUE constraint decides and the API returns 409. The `FOR NO KEY UPDATE` lock on upload is the weakest lock that serialises two uploads by the same candidate (only one can become primary) without blocking FK checks from new applications.

9.6 Vector implementation

Question (PDF §8)	Answer
What is embedded	Resume: the extracted text of each uploaded PDF/TXT. Job: `title + description + "Required skills: …"` (only required skills)
Model	`sentence-transformers/all-MiniLM-L6-v2`, 384 dimensions, run locally in Node.js with `@huggingface/transformers` (no external API, resumes never leave the server)
Chunking	Up to 254 word-piece tokens per chunk, 32-token overlap, words never split, at most 40 chunks. Each chunk: mean pooling + L2 normalisation. Document vector: token-weighted mean of the chunks, L2-normalised again

Question (PDF §8)	Answer
Storage	`resume_embeddings` / `job_embeddings`: `vector(384)`, PK = FK CASCADE, CHECK unit length, `model_name`, `chunk_count`, `source_hash` (SHA-256 of the text, detects stale vectors)
Index	HNSW with `vector_cosine_ops` on both tables
Metric	Cosine similarity `1 - (a <=> b)`; `<=>` is pgvector's cosine distance
Retrieval	`rank_candidates_for_job()`: read the job vector once, `ORDER BY re.embedding <=> v_job_vector LIMIT k` over primary resumes of active users (optionally applicants only), then add skill counts. `compute_match_score()` stores the score on each application

Excerpt (migration 015):

```
SELECT r.candidate_id, r.resume_id, re.embedding <=> v_job_vector AS distance
  FROM resume_embeddings re
  JOIN resumes r ON r.resume_id = re.resume_id
  JOIN users   u ON u.user_id   = r.candidate_id
 WHERE r.is_primary AND u.is_active
   AND (NOT p_applicants_only OR EXISTS (SELECT 1 FROM applications a
                                         WHERE a.job_id = p_job_id AND a.candidate_id =
r.candidate_id))
 ORDER BY re.embedding <=> v_job_vector
 LIMIT p_limit
```

Real result for the *Machine Learning Engineer* job: Aditi Sharma 0.7819 (3/3 skills), Rohan Verma 0.6658 (1/3 skills), Divya Krishnan 0.5032 … Sneha Kulkarni (frontend) 0.2438 last. Rohan ranks second with only one listed skill at the required level because his resume describes deep learning in TensorFlow — exactly what keyword matching misses. With 12 resumes the planner prefers an exact sequential scan over the HNSW index (cheaper for a tiny table); the index is used automatically as data grows (shown with `EXPLAIN ANALYZE` in vector-design.md §8).

10 Important queries

`db/queries/important_queries.sql` holds 41 commented queries (each with DEMONSTRATES / RETURNS / WHY). The file is read-only: the queries that write do so inside transactions they roll back. Twelve representative ones:

Q3 — Jobs with at least 3 applicants (JOIN, GROUP BY, HAVING). HAVING filters groups after aggregation, which WHERE cannot do. Used to prioritise screening.

```
SELECT c.name AS company, j.title, count(a.application_id) AS applicants
FROM jobs j JOIN companies c ON c.company_id = j.company_id
            JOIN applications a ON a.job_id = j.job_id
GROUP BY c.name, j.job_id, j.title
HAVING count(a.application_id) >= 3
ORDER BY applicants DESC, j.title;
```

Q8 — Required-skill coverage per applicant (correlated subqueries). The inner queries use `a.job_id` and `a.candidate_id` from the outer row; this is the explainable "skills 3/3" badge next to the semantic score.

```
SELECT j.title, u.full_name, a.status,
       (SELECT count(*) FROM job_skills js
          JOIN candidate_skills cs ON cs.skill_id = js.skill_id AND cs.candidate_id = a.candidate_id
                                  AND cs.proficiency >= js.min_proficiency
         WHERE js.job_id = a.job_id AND js.is_required) AS required_met,
       (SELECT count(*) FROM job_skills js WHERE js.job_id = a.job_id AND js.is_required) AS
required_total
FROM applications a JOIN jobs j ON j.job_id = a.job_id JOIN users u ON u.user_id = a.candidate_id;
```

Q9 — Candidates with above-average experience (scalar subquery). The subquery returns one value that the outer WHERE compares against.

```
SELECT u.full_name, cd.years_experience
FROM candidates cd JOIN users u ON u.user_id = cd.user_id
WHERE cd.years_experience > (SELECT avg(years_experience) FROM candidates)
ORDER BY cd.years_experience DESC;
```

Q10 — Skill demand vs supply (COUNT DISTINCT over two M:N tables + HAVING). `DISTINCT` is needed because joining two junction tables multiplies rows.

```
SELECT s.name, count(DISTINCT js.job_id) AS jobs_requiring, count(DISTINCT cs.candidate_id) AS
candidates_with_skill
FROM skills s
JOIN job_skills js ON js.skill_id = s.skill_id AND js.is_required
LEFT JOIN candidate_skills cs ON cs.skill_id = s.skill_id
GROUP BY s.skill_id, s.name
HAVING count(DISTINCT js.job_id) >= 2;
```

Q11 — Average days between stages (window function LAG over the audit table). `LAG` reads the previous history row of the same application, giving "time to shortlist" and "time to offer".

```
WITH steps AS (
  SELECT old_status, new_status,
         changed_at - lag(changed_at) OVER (PARTITION BY application_id ORDER BY changed_at) AS took
  FROM application_status_history)
SELECT old_status || ' -> ' || new_status AS transition, count(*),
       round(avg(extract(epoch FROM took) / 86400)::numeric, 1) AS avg_days
FROM steps WHERE old_status IS NOT NULL GROUP BY old_status, new_status ORDER BY avg_days;
```

Q16 — Views and the report function. Shows that a view is queried like a table and a set-returning function is used in FROM.

```
SELECT * FROM company_hiring_summary;
SELECT job_title, total_applications, shortlist_rate_pct
  FROM generate_hiring_pipeline_report((SELECT company_id FROM companies WHERE name = 'Nimbus Analytics
Pvt Ltd'));
```

Q20 — Semantic ranking with pgvector. Top 5 candidates for the ML job by cosine similarity, with skill counts.

```
SELECT full_name, similarity, matched_skills, required_skills, has_applied
FROM rank_candidates_for_job((SELECT job_id FROM jobs WHERE title = 'Machine Learning Engineer'), 5,
FALSE);
```

Q22 — Reverse k-NN: best open jobs for one candidate. The raw `ORDER BY embedding <=> vector LIMIT k` shape that the HNSW index serves, combined with a relational filter (`status = 'open'`).

```
SELECT j.title, round((1 - (je.embedding <=> q.v))::numeric, 4) AS similarity
FROM job_embeddings je
JOIN jobs j ON j.job_id = je.job_id AND j.status = 'open'
CROSS JOIN (SELECT re.embedding AS v FROM resume_embeddings re
              JOIN resumes r ON r.resume_id = re.resume_id AND r.is_primary
              JOIN users u ON u.user_id = r.candidate_id
             WHERE u.email = 'aditi.sharma@mail.example') q
ORDER BY je.embedding <=> q.v LIMIT 3;
```

Q23 — Apply-to-job transaction, step by step. `BEGIN` , `FOR SHARE` , `INSERT … SELECT … RETURNING` with `compute_match_score()` ; inside the transaction the trigger-updated count is 4, after `ROLLBACK` it is 3 again.

Q24 — Skill gap with EXCEPT. Missing skills are the job's required skills minus the candidate's skills.

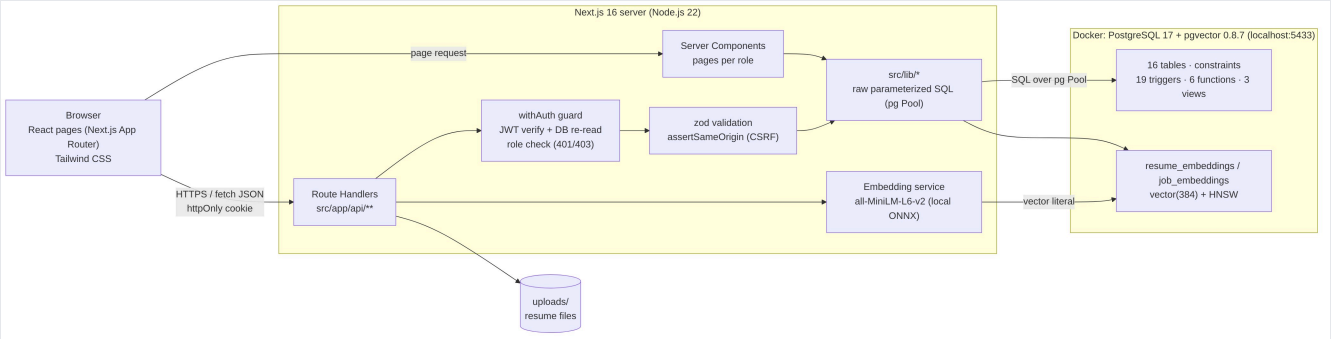
```
SELECT js.skill_id FROM job_skills js WHERE js.job_id = $job AND js.is_required
EXCEPT
SELECT cs.skill_id FROM candidate_skills cs WHERE cs.candidate_id = $candidate;
```

Q28 — Atomicity proof. Inside one block, `update_application_status(…, 'offered')` succeeds and a second offer then violates `UNIQUE (application_id)` ; the status change, its trigger-written history row and its notification are all undone (`before` = `after`).

Q37 — Per-company dashboard table. LEFT JOIN to the `company_hiring_summary` view (so companies hidden by its HAVING still appear), pre-aggregated derived tables for offers and ratings (joining them directly would multiply rows), `nullif` to avoid division by zero, and a scalar subquery for the "above average" flag. Acceptance rate = accepted ÷ (accepted + declined).

The full list (Q1–Q41) with every concept is in the file itself; Q41 shows `ON DELETE RESTRICT` refusing to delete a company that still has recruiters and jobs.

11 System architecture



System architecture, rendered from the Mermaid source in `docs/project-report.md`.

Layers:

1. **Presentation** — React pages for each role (`src/app/candidate`, `recruiter`, `admin`, `notifications`), components in `src/components`. The UI may hide buttons, but it is never the protection.
2. **API** — Next.js Route Handlers under `src/app/api` (36 route files). Every protected handler is wrapped in `withAuth({ roles })`.
3. **Business logic and data access** — `src/lib` (applications, offers, interviews, admin, recruiter, candidate). Raw SQL through one `pg` connection pool; every value is a `$n` parameter. No ORM, so every query is visible for review.
4. **Database** — PostgreSQL + pgvector. It is the final authority: constraints and triggers enforce the rules even if the API had a bug.
5. **Embeddings** — the model runs in the same Node.js process, loaded once and cached; files are stored under `uploads/`.

Request flow — a candidate applies for a job:

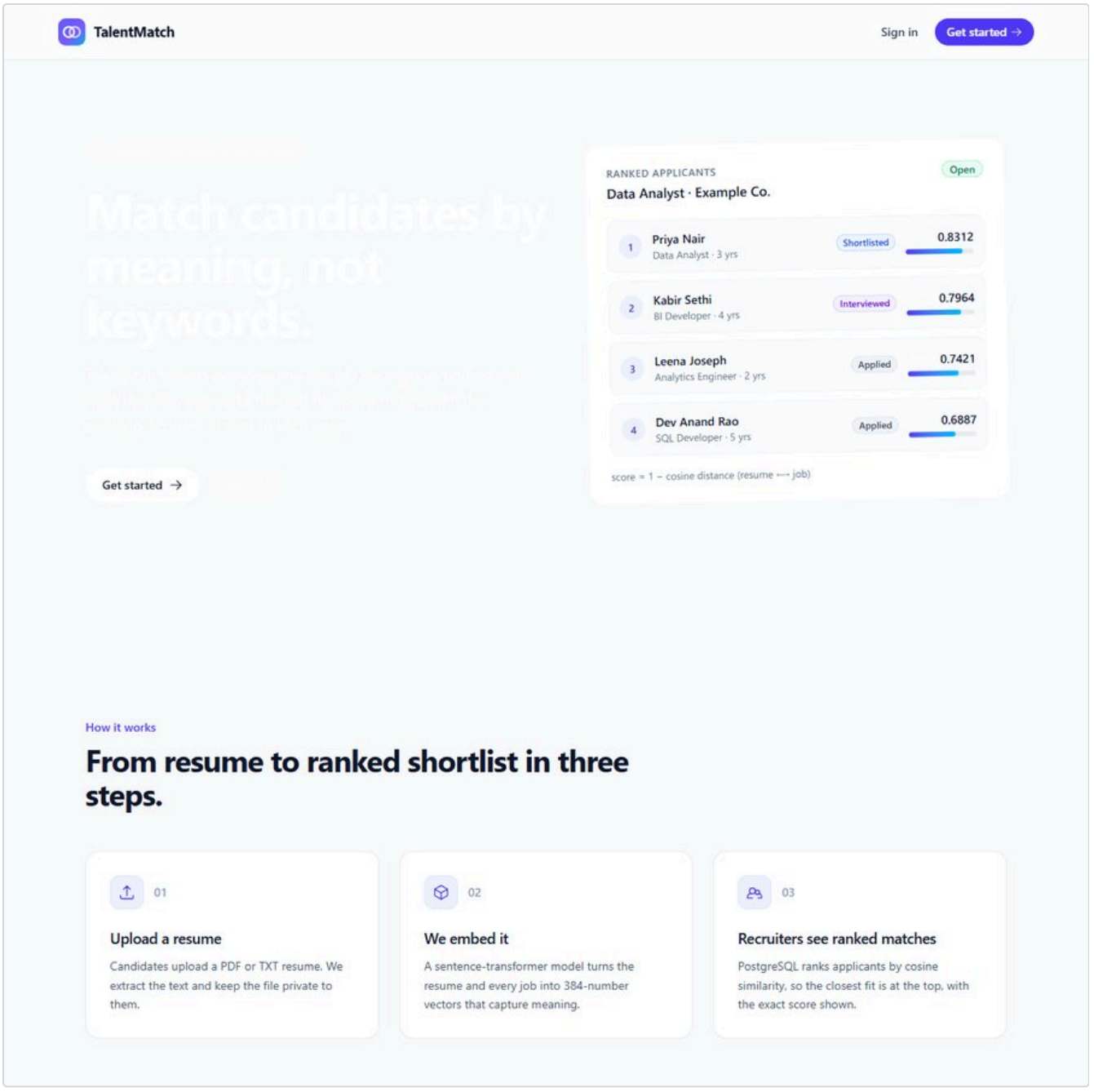
```
Browser                          Next.js route handler                          PostgreSQL
POST /api/candidate/applications ──►
 cookie tm_session=<JWT>         withAuth: verify JWT (HS256, iss, aud, exp)
 {jobId, resumeId}               SELECT user FROM users WHERE user_id=$1   ──►  (inactive / role
changed → 401)
                                 role must be candidate                         (else 403)
                                 assertSameOrigin, zod applicationCreateSchema  (else 403 / 400)
                                 BEGIN                                      ──►
                                 SELECT … FROM jobs … FOR SHARE                  draft → 404, closed →
409
                                 SELECT … FROM resumes … AND candidate_id=$2     not theirs → 404
                                 INSERT INTO applications … compute_match_score  triggers: initial
status,
                                                                                 applicant_count +1,
history row
                                 INSERT INTO notifications (recruiter)
                                 COMMIT                                     ──►  (UNIQUE violation →
ROLLBACK → 409)
                        ◄──      201 { application: { id, status, matchScore } }
```

Configuration comes only from environment variables (`DATABASE_URL`, `AUTH_JWT_SECRET`, optional `AUTH_SESSION_MAX_AGE_SECONDS`) in `.env.local`, which is gitignored; `.env.example` holds only names and placeholders.

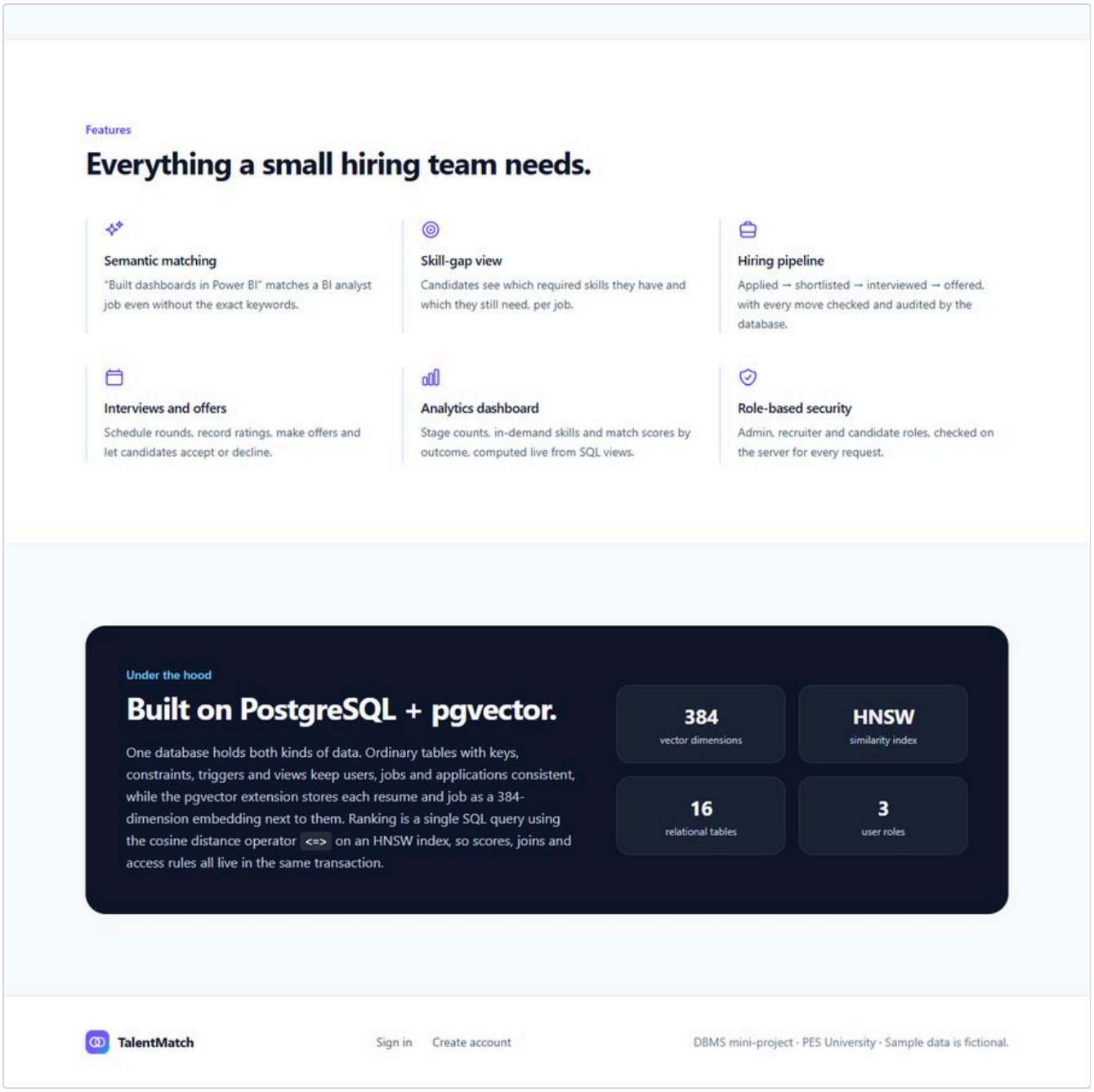
12 Application screenshots

All screenshots are in `docs/screenshots/final/` and were taken from the final build with the seeded demo data. Very tall pages are split across two images or shown from the top; the full images are in the repository.

Landing page

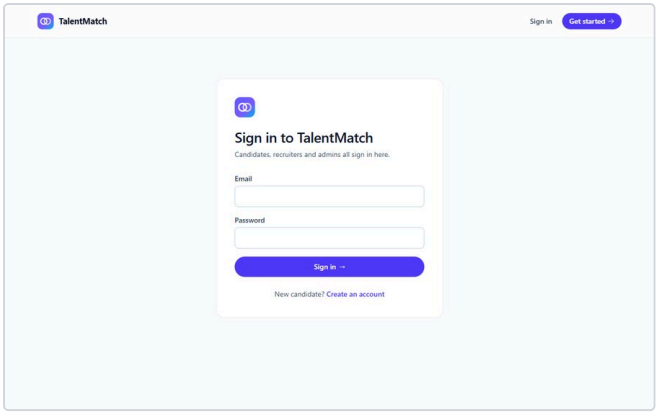


31. Landing page (desktop) (part 1 of 2)

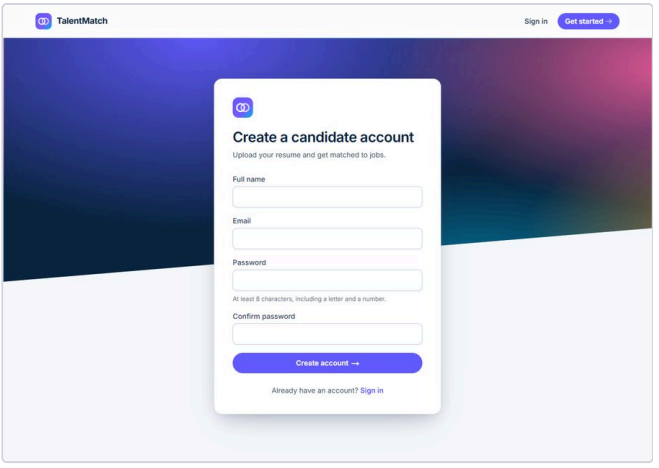


31. Landing page (desktop) (part 2 of 2)

Authentication

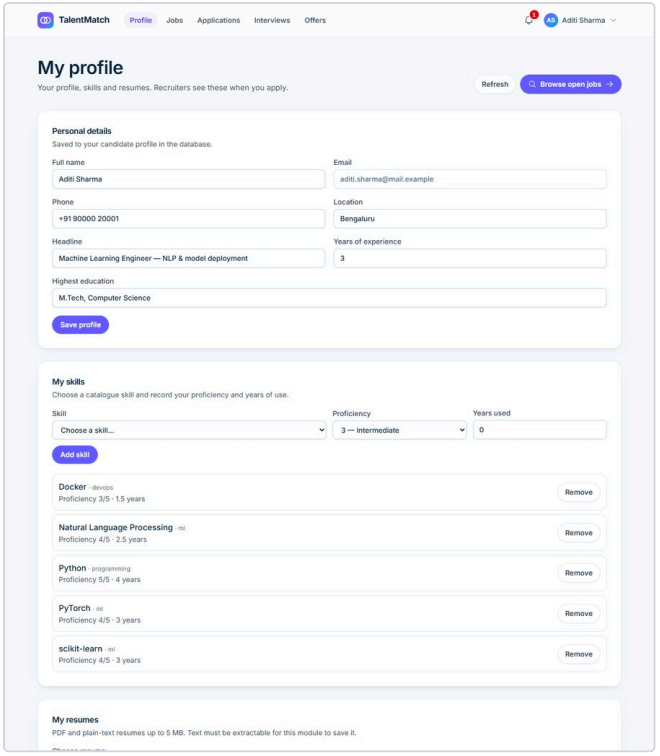


1. Login

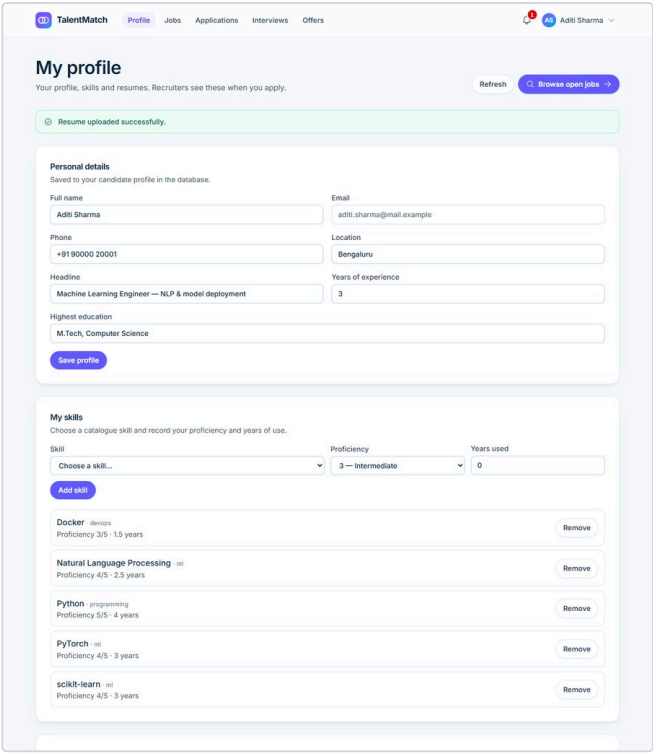


2. Candidate registration

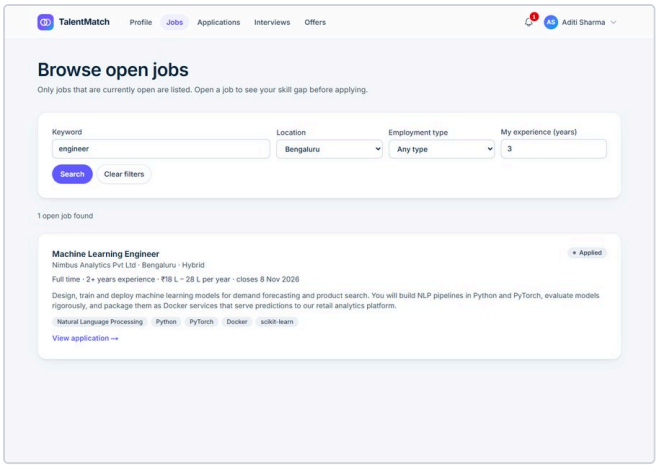
Candidate



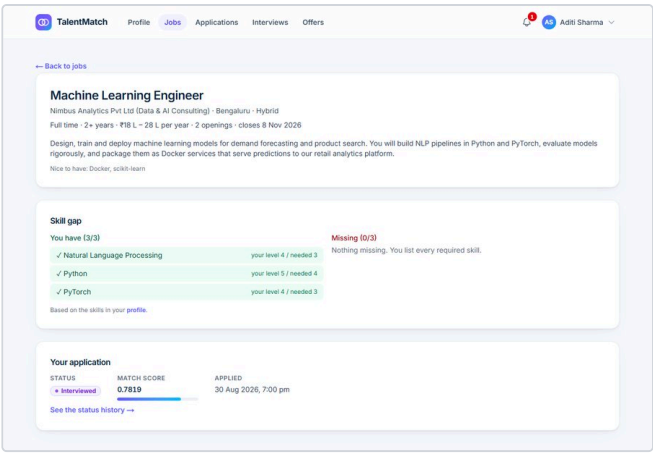
3. Profile and skills (top of page)



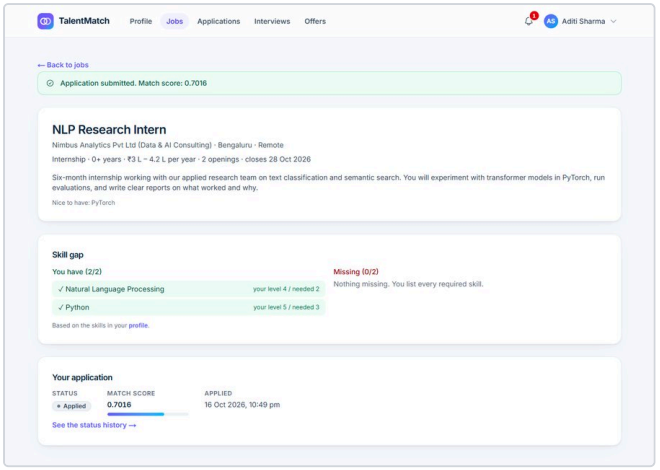
4. Resume upload (top of page)



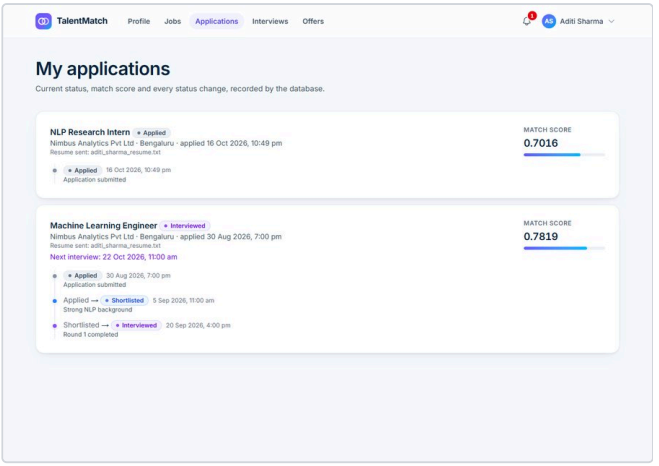
5. Job search and filters



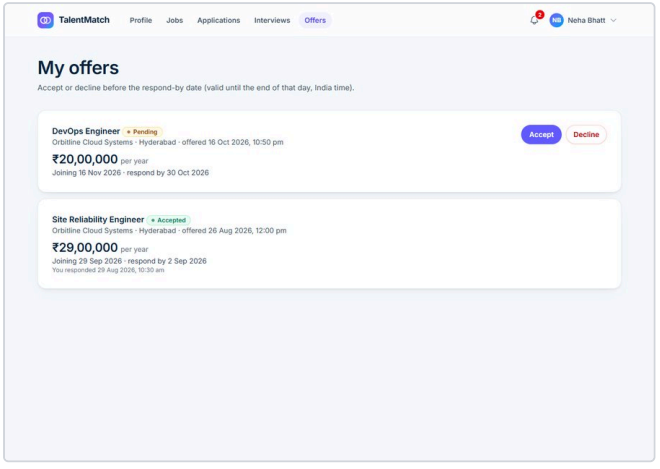
6. Skill gap (EXCEPT)



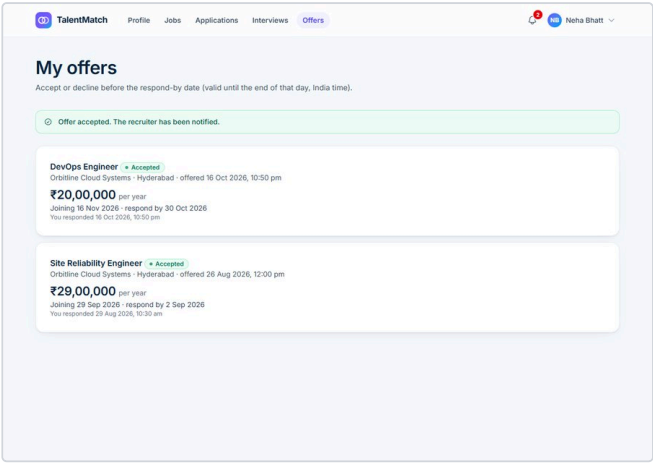
7. Apply with match score



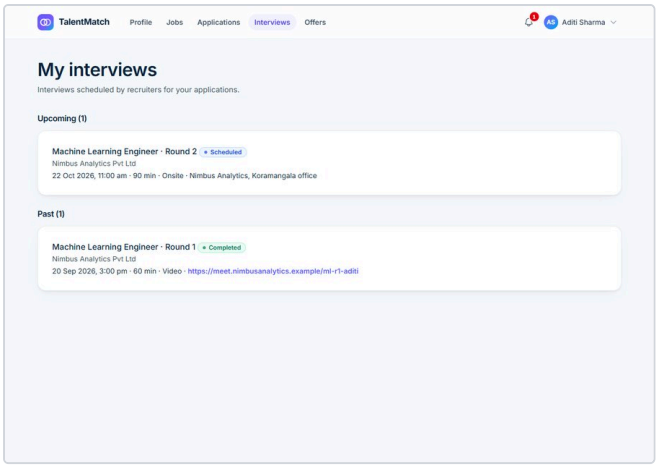
8. My applications with history



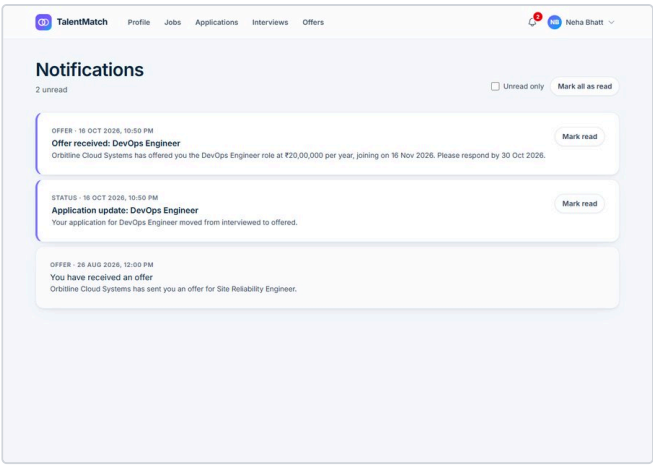
20. Pending offer



21. Offer accepted

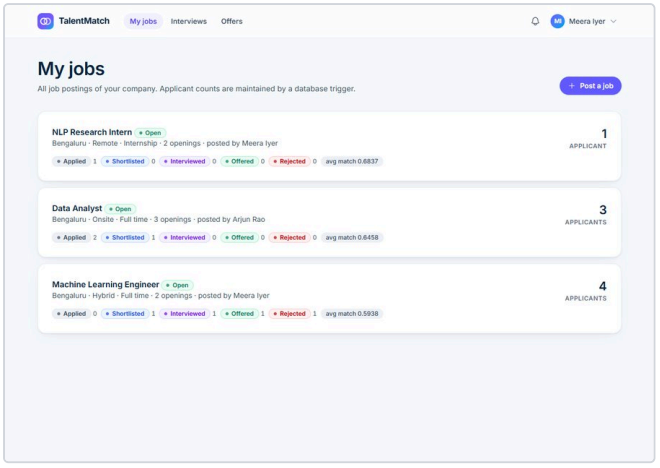


22. Candidate interviews

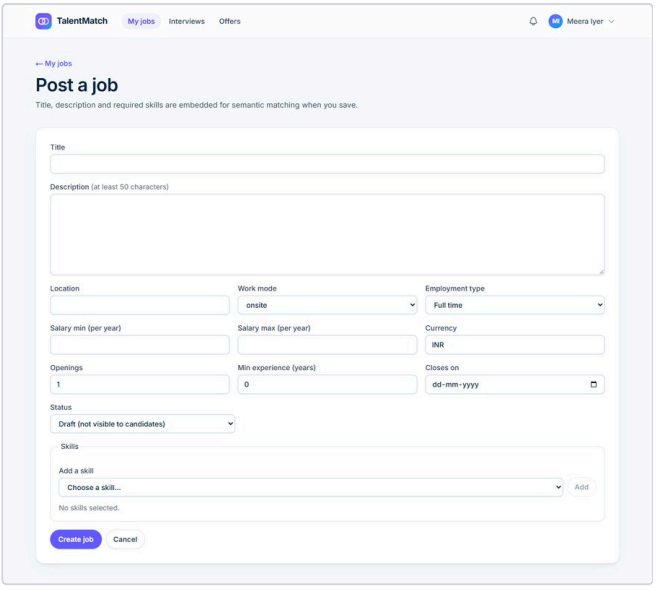


23. Candidate notifications

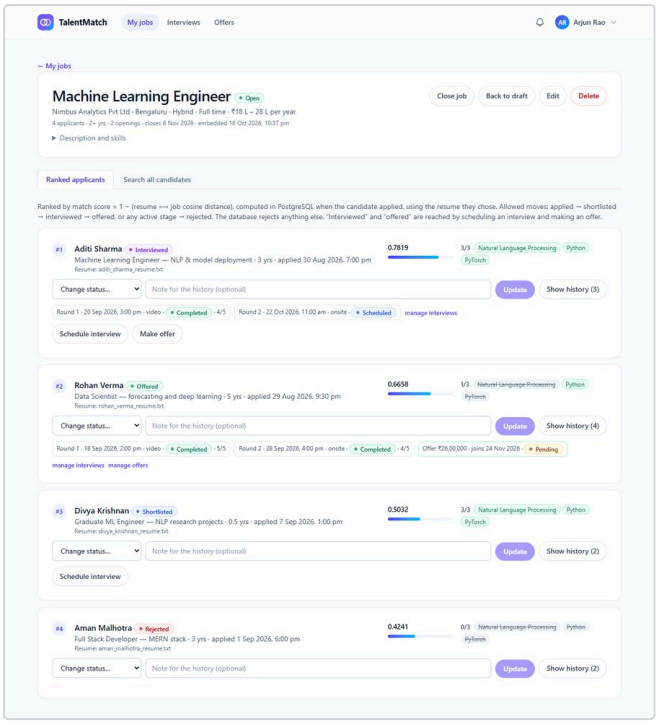
Recruiter



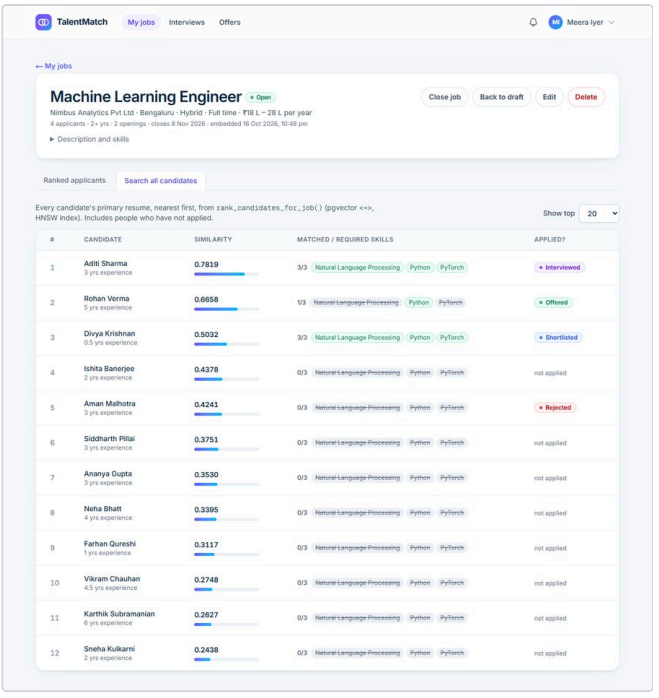
9. My jobs



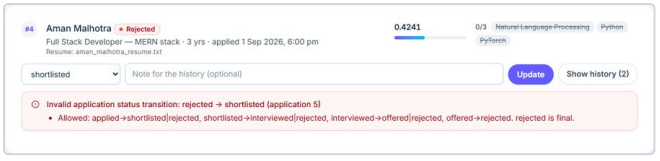
10. Job form with required skills



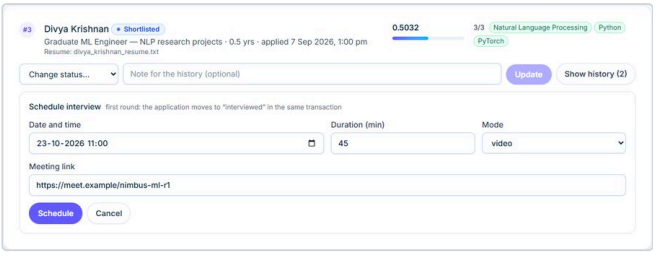
11. Applicants ranked by cosine score



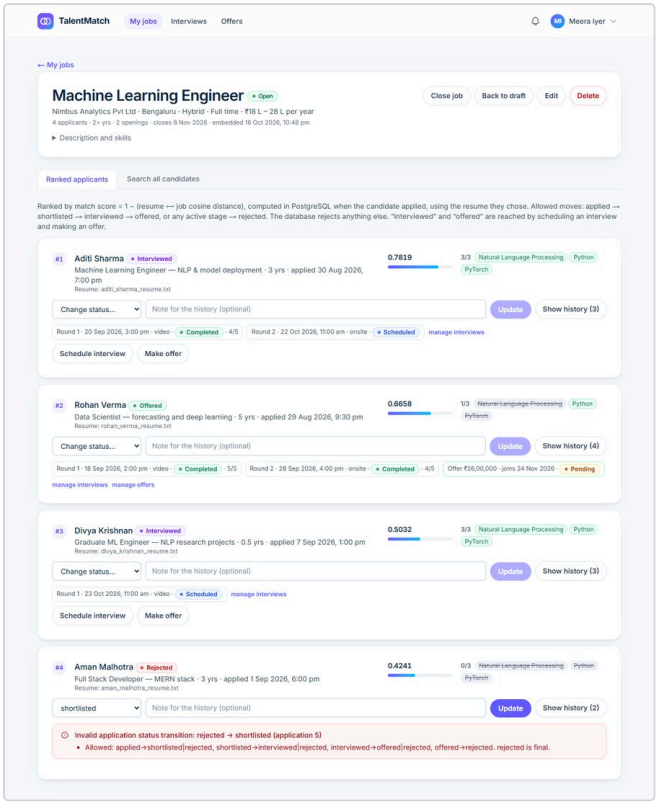
12. Semantic search over all candidates



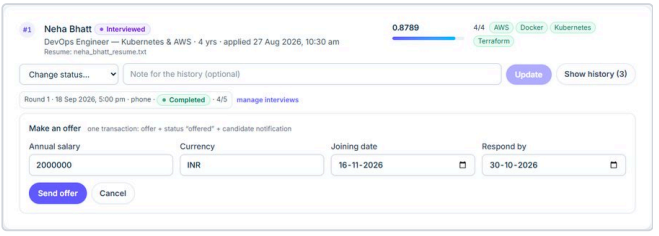
13. Invalid transition refused by the trigger



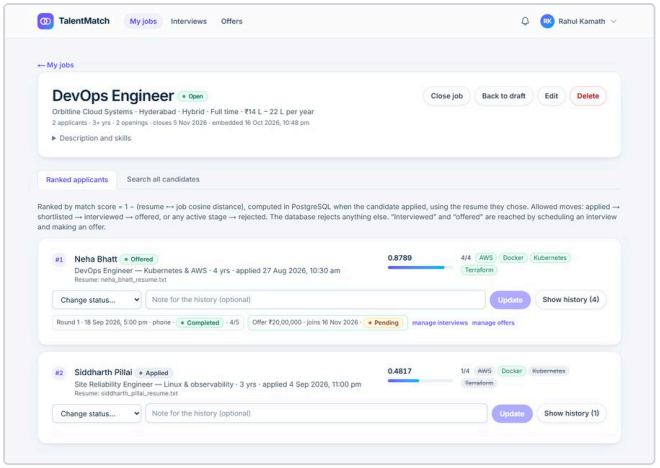
14. Interview scheduling



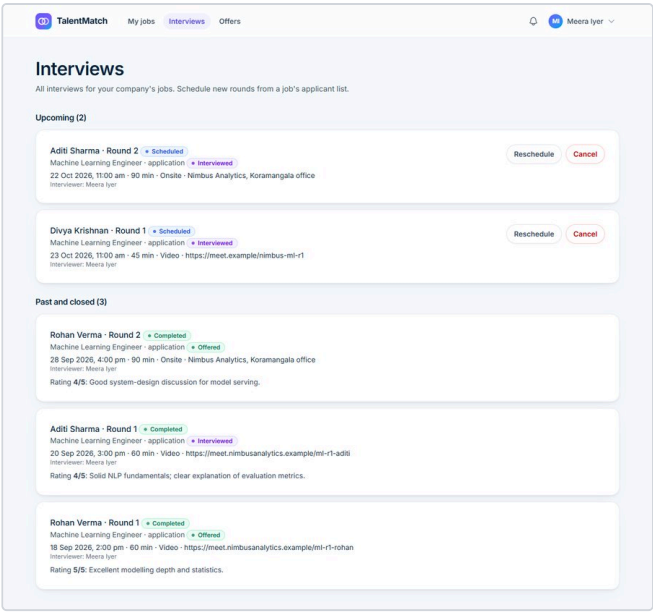
15. Interview scheduled



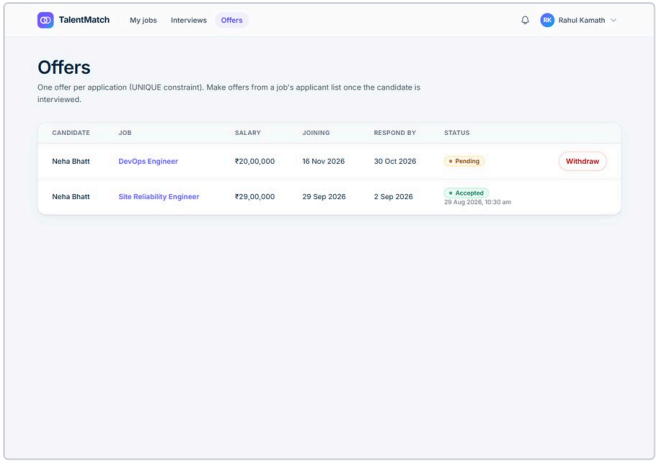
16. Make offer



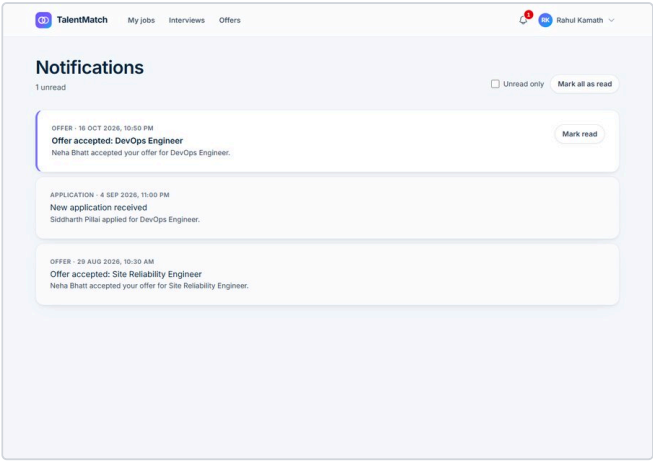
17. Offer made



18. Company interviews

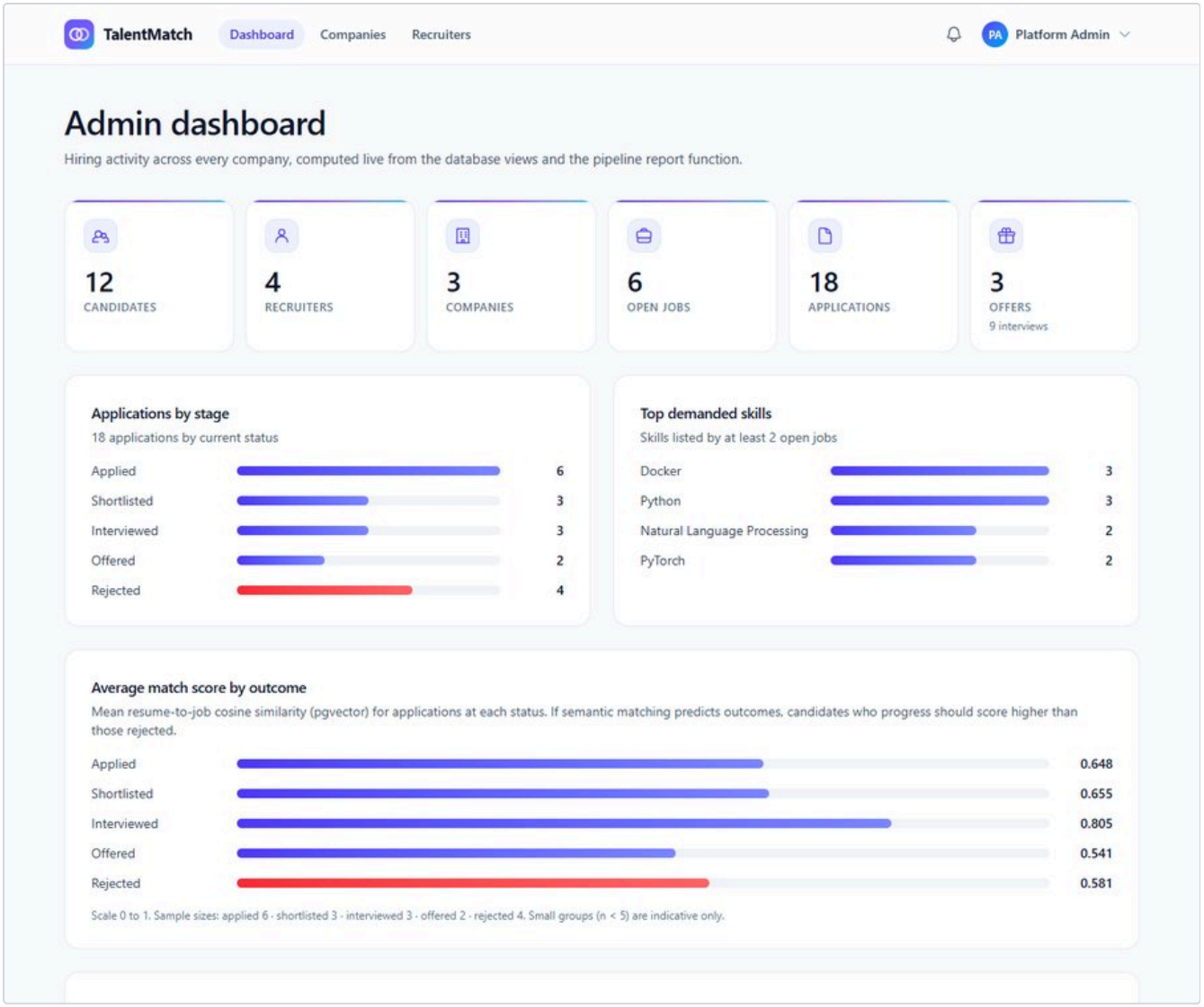


19. Company offers

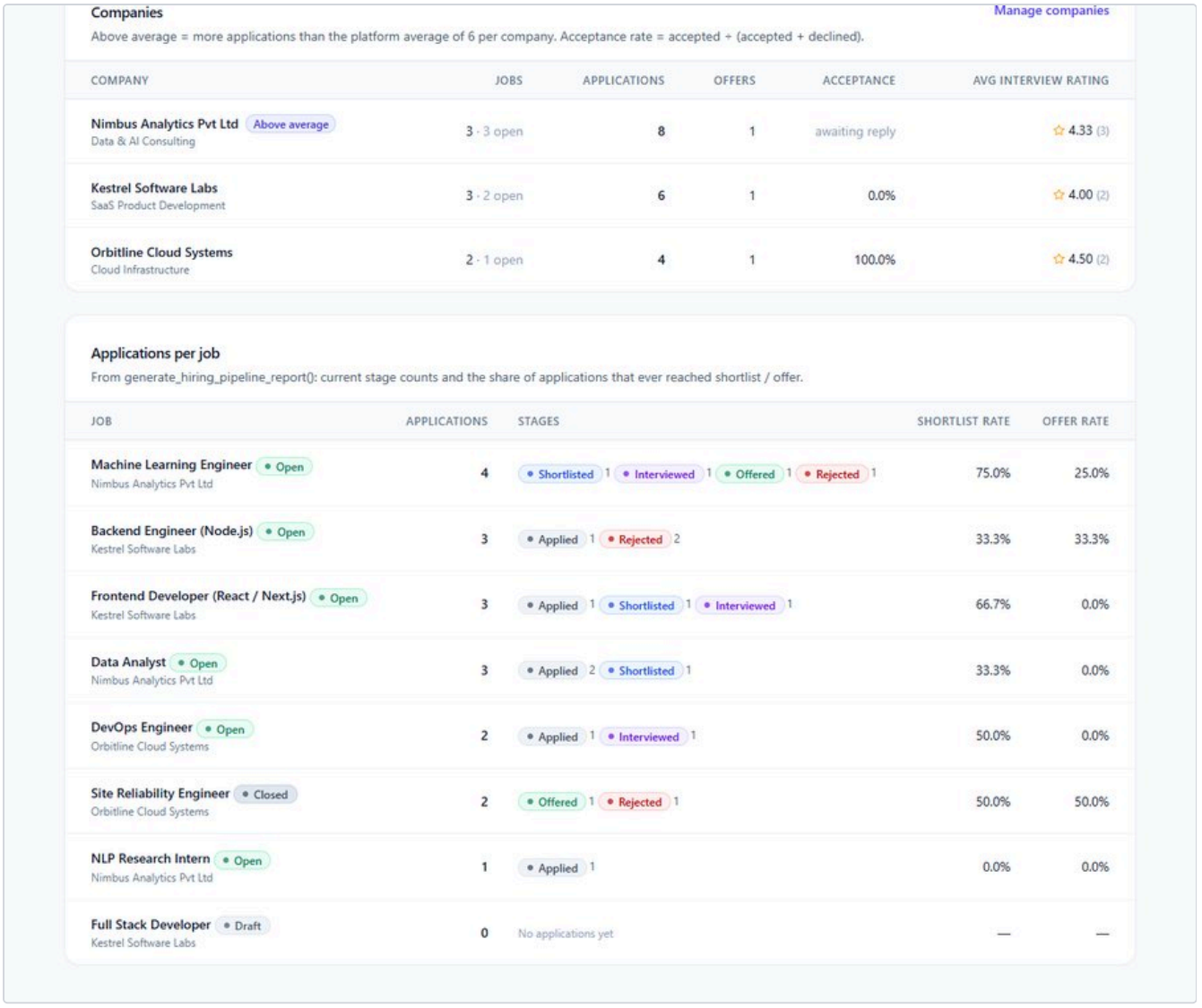


24. Recruiter notifications

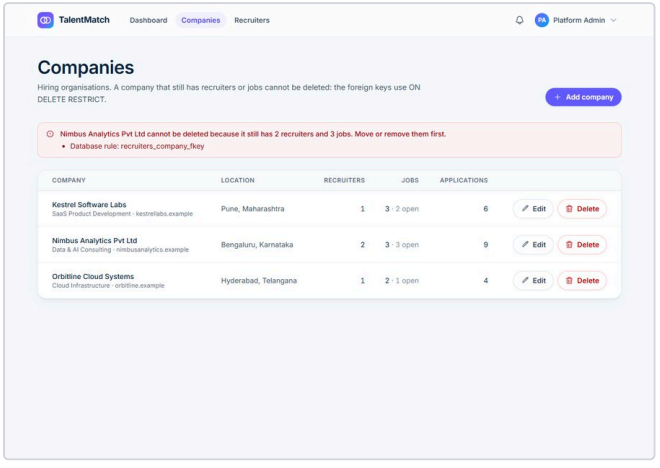
Admin



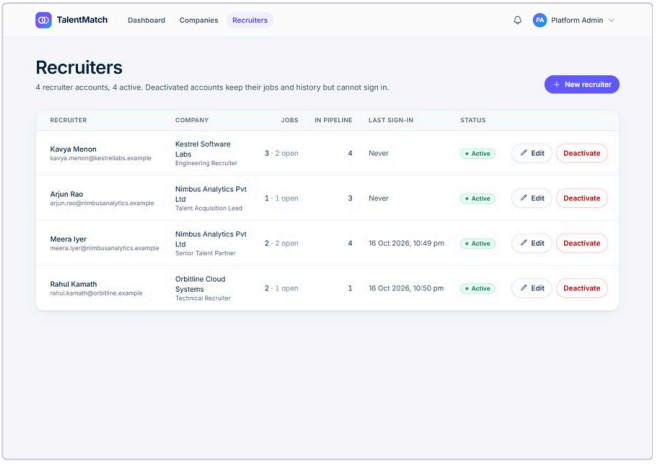
25. Dashboard (views + report function) (part 1 of 2)



25. Dashboard (views + report function) (part 2 of 2)

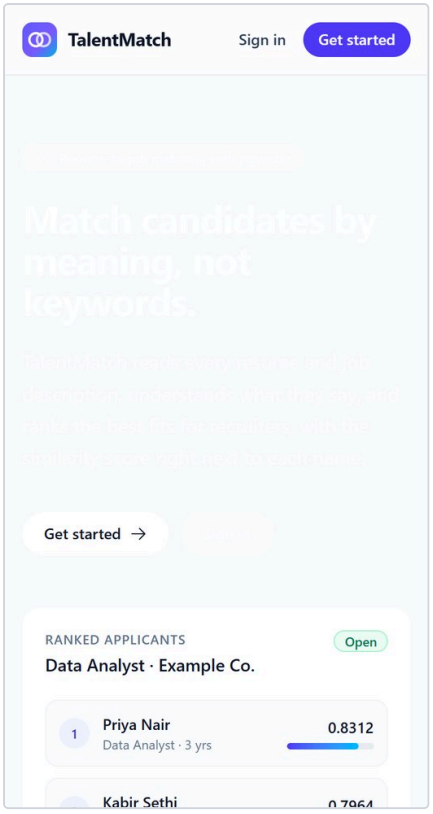


26. Company delete refused (ON DELETE RESTRICT)

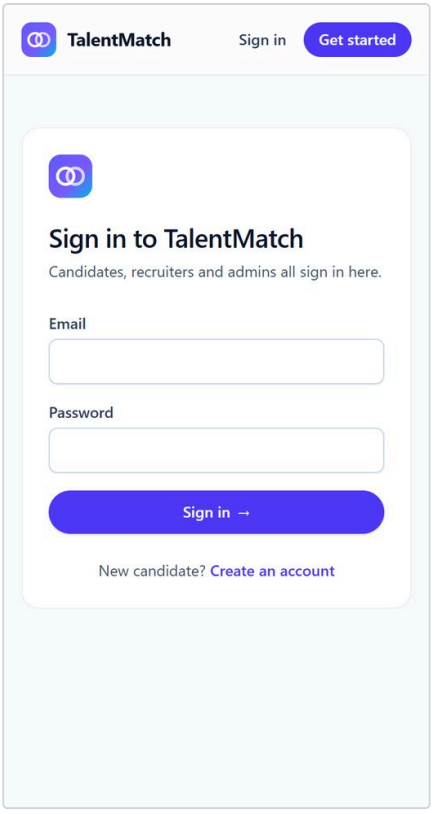


27. Recruiter accounts and workload

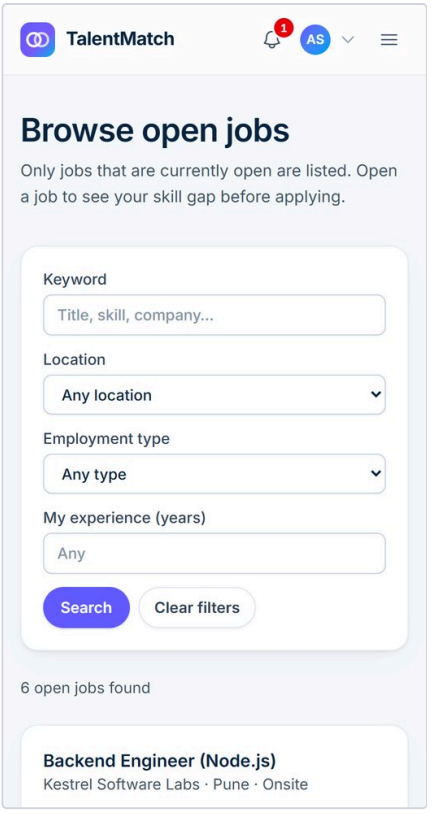
Mobile layout



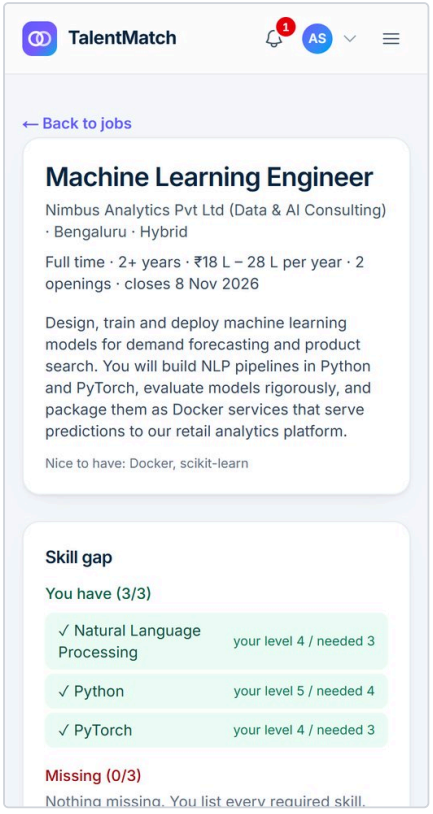
32. Landing page (top of page)



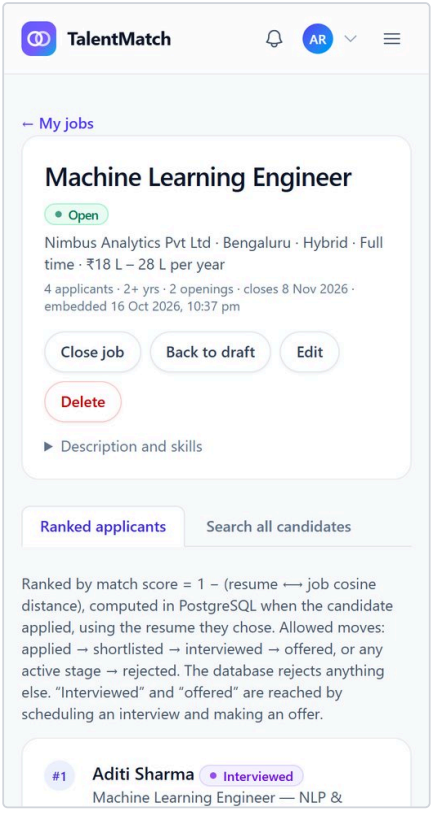
33. Sign in (top of page)



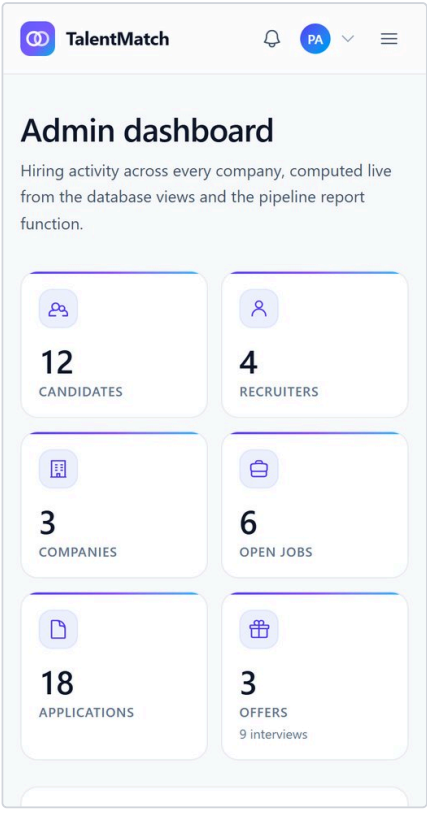
28. Candidate jobs (top of page)



34. Job detail and skill gap (top of page)



29. Ranked applicants (top of page)



30. Admin dashboard (top of page)

13 Testing results

Final run on 16 Oct 2026 — Windows 11, Node.js 22.19.0, Next.js 16.4.0 production build, PostgreSQL 17.11 + pgvector 0.8.7 in Docker, database rebuilt from scratch with migrations 001–024. The full table of individual cases with expected and actual results is in test-cases.md.

Suite	Command	Result
Migrations	`npm run db:reset -- --yes` then `npm run db:migrate`	`Done: 24 applied, 0 already up to date.`
Seed + embeddings	`npm run db:seed`, `npm run db:embed`	17 users, 18 applications, 40 history rows, 9 interviews, 3 offers, 10 notifications; 12 resumes + 8 jobs embedded
Build	`npm run build`	exit 0, compiled successfully (1 known Turbopack tracing warning)
Unit tests	`npm run test:unit`	**85 tests, 25 suites, 85 pass, 0 fail**
Integration tests	`npm run test:integration`	**147 tests, 35 suites, 147 pass, 0 fail** (96 s); 0 `@test.example` users left afterwards
Constraint tests (SQL)	`db/tests/stage1_constraint_tests.sql`	8 of 8 invalid writes rejected by the intended constraint, each rolled back
Advanced-feature tests (SQL)	`db/tests/stage2_advanced_tests.sql`	27 PASS lines (T1–T13), "All Stage 2 tests passed"
Important queries	`db/queries/important_queries.sql` with `ON_ERROR_STOP`	Q1–Q41 ran, exit code 0
Transaction demo	`db/demos/transaction_demo.sql`	COMMIT part persisted then cleaned up, ROLLBACK and SAVEPOINT parts undone; Data Analyst `applicant_count` 3 before and 3 after

What the tests cover:

- **Unit (85):** bcrypt and JWT handling, cookie flags, every zod schema, chunking and vector maths, the real embedding model (384 dimensions, unit length, deterministic, ML text closer to an ML job than React text), login rate limiter on a fake clock, date formatting, upload checks.
- **Integration:** the real API on `next start` against the real database and model — registration, login, forged and expired tokens, 401/403 for every role on every module, ownership (another user's record → 404, another company's → 403), file upload limits, the apply transaction and its forced rollback, every status transition rule, interviews and offers with three forced rollbacks, the deferred trigger, notifications, the admin dashboard recomputed from base tables with independent SQL, security headers, CSRF, rate limiting.
- **SQL suites:** each invalid write is rejected by the intended constraint or trigger inside a rolled-back transaction; triggers, functions and views are checked against base-table counts.
- **Negative controls:** several fixes were removed on purpose and the tests were run again to prove they fail without them (e.g. dropping the withdraw-offer trigger made exactly its two tests fail).

Consistency after the run: 0 test users left, 18 applications and 3 offers (the seed), `applicant_count` equal to the real count for every job, every stored `match_score` equal to `1 - (resume <=> job)` within the 4-decimal rounding.

Bugs found by testing (all fixed and covered by tests): PDF uploads failing in the production build, a race that could create two primary resumes, offers withdrawable twice, a pending offer left on a rejected application, an interview completable before it happened, rejection possible after an accepted offer, an offer possible while an interview was still scheduled, no security headers, CSRF check ignoring `Sec-Fetch-Site`. See test-cases.md § Bugs found by tests.

14 Conclusion and future enhancements

Conclusion

TalentMatch shows that one PostgreSQL database can hold both the structured hiring workflow and the unstructured text it depends on. The relational part keeps the workflow correct: 57 CHECK constraints, 22 foreign keys with deliberate delete rules, 19 triggers including a deferred constraint trigger, six functions, three views and eight multi-statement transactions mean that an invalid state is refused by the database itself, whether the change comes from the web app or from psql. The vector part adds what SQL alone cannot: a candidate's resume is ranked by meaning, and the score shown to recruiters is a real cosine similarity that can be recomputed and checked in SQL. The Next.js application exposes all of this to three roles through a server-side API with authentication, role checks, ownership checks in SQL and parameterized queries. The project goes beyond CRUD: most of its value is in the rules, ranking and analytics, and every claim is backed by automated tests.

Limitations

- Document vectors are a mean of chunk vectors, so a long resume with mixed content gets a blurred vector; the model is English-centric.
- Similarity scores are relative (0.5 is not "50 % qualified"); with only 18 seeded applications, the dashboard's "score by outcome" is indicative only.
- The HNSW index is not used by the planner on 12 resumes (a sequential scan is cheaper and exact); its benefit appears only with larger data.
- Login rate limiting is kept in the memory of one server process; the client IP comes from `X-Forwarded-For` and is only reliable behind a proxy.
- Stateless JWT: a copied token stays valid until it expires (8 h) unless the account is disabled or its role changes.
- No email/SMS, password reset or email verification; no `script-src` Content-Security-Policy.

Future enhancements

1. Per-chunk embeddings with "best chunk wins" scoring and an explanation of *why* a candidate matched (which resume passage matched which requirement).
2. Ranking jobs for a candidate as a function and API ("recommended jobs"); today it exists as query Q22.
3. Full-text search (`tsvector` + GIN) combined with vector search (hybrid ranking).
4. Email/SMS notifications for interviews and offers; calendar invites.
5. Integration with external job boards and with onboarding/payroll for hired candidates (from the synopsis).
6. Shared rate-limit store (database table or Redis), token revocation list, HSTS and a nonce-based CSP behind an HTTPS proxy.
7. Partitioning `application_status_history` and `notifications` by date once they grow large.

15 Team contribution

Member	SRN	Contribution recorded in git
Rohith Karning	PES1UG24AM229	106 commits on this branch before the final documentation commits: project setup, schema and migrations, seed data, triggers, functions and views, authentication and RBAC, semantic matching with pgvector, recruiter module, interviews/offers/notifications, admin dashboard, security hardening, tests and documentation
Saanvi Maharana	PES1UG24AM237	1 commit (`945c8e2`, 10 Oct 2026): first version of the candidate module — profile, skills and resume API routes, candidate dashboard component, candidate integration tests

Division of responsibilities:

Member	Responsibilities
Rohith Karning	Project setup; schema and migrations; advanced SQL (triggers, functions, views, transactions); authentication; semantic matching with pgvector; recruiter module; interviews and offers; admin dashboard; security hardening; documentation
Saanvi Maharana	Candidate module: profile, skills and resume upload (API routes and UI), and the first candidate integration tests

A Requirements compliance matrix

Each requirement of the course handout (DBMS Experiential Learning – Level 3 Project Requirements) mapped to where this report and repository satisfy it.

Handout requirement	How TalentMatch meets it	Section
§1 Unique title; meaningful problem; not simple CRUD	Semantic resume–job ranking plus a rule-enforced hiring pipeline and analytics	1, 14
§2 Two-member team with defined responsibilities	Rohith Karning (PES1UG24AM229), Saanvi Maharana (PES1UG24AM237)	15
§3 SQL (mandatory) + NoSQL or Vector database	PostgreSQL 17 + pgvector 0.8.7 (Option 2)	7
§4 ER diagram → relational schema; entities, attributes, PK, FK, relationships, constraints; vector data model	ER diagram with all attributes; 16-table schema; embeddings, metadata (model_name, chunk_count, source_hash), HNSW index and retrieval strategy documented	5, 6, 8, 9.6, B
§5 DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Migrations 001–024; ALTER in 009 and 018; DROP in `db/reset.sql`; 46 indexes; 3 views	9
§6 DML: INSERT, UPDATE, DELETE, SELECT with JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	Used across the API; 41 documented queries including correlated and scalar subqueries, window functions, EXCEPT and CTEs	9, 10
§7 At least four advanced SQL concepts and one NoSQL/vector	Transactions, triggers, stored functions, views, indexing (B-tree, partial, HNSW) + vector similarity search	9.2–9.6
§8 Vector search: data embedded, model, storage, metric, retrieval	Resume text and job text; all-MiniLM-L6-v2 (384-d, local); `vector(384)` tables with HNSW; cosine similarity; `rank_candidates_for_job()`	9.6
§9 Functional Next.js interface: forms, tables, search, filtering, dashboards	Role-based portals for candidate, recruiter and admin; job search and filters; admin dashboard	4, 12
§10 Backend/API layer; no hard-coded credentials; environment variables	Next.js Route Handlers (36 route files); `DATABASE_URL` and `AUTH_JWT_SECRET` from `.env.local` (git-ignored)	11
§11 Authentication; at least 2 roles	JWT in signed httpOnly cookie, re-checked against the database each request; 3 roles	4, 11
§12 Validation at database level; data consistency and integrity	57 CHECK, 22 FK with chosen delete rules, UNIQUE, NOT NULL, 19 triggers, transactions	6, 8, 9
§13 Security; no plain-text passwords or credentials	bcrypt hashes (a CHECK rejects non-bcrypt values), parameterised SQL, CSRF check, rate limiting, server-side role and ownership checks	9.1, 11
§14 Documentation (14 items)	Sections 1–14 of this report, in the handout's order	1–14
§15 Git/GitHub; individual commits; README	Repository with README (setup and run); feature branches merged by pull request; commit record in Section 15	15
§16 Demonstration items	Problem, ER and schema, implementation, queries, interface, integration, advanced features, test cases and contributions are each covered by a section of this report	1–15

§17 Originality	Own problem statement (synopsis), schema and implementation	1, 5–9
§18 Level 3: beyond CRUD; justified choice of SQL vs vector per type of data	Per-data justification table and reasons for pgvector in the same database	7

B Full data dictionary

Reproduced from `docs/data-dictionary.md` in the repository.

This file was written by hand from the live database (`information_schema.columns`) after running migrations 001–014, so it matches the real schema. Triggers, functions and views are described in advanced-features.md.

Legend: PK primary key · **FK** foreign key · **UK** unique · **NN** NOT NULL · **ID** `GENERATED ALWAYS AS IDENTITY` (auto-number)

All timestamps are `TIMESTAMPTZ` (stored in UTC, shown in local time). Money is `NUMERIC(12,2)` (exact, never floating point).

---

1. `users`

Every login account. Its role decides which profile table (if any) holds the extra details.

Column	Type	Null	Default	Constraints / notes
user_id	BIGINT	NN	ID	**PK**
email	VARCHAR(255)	NN		**UK** `users_email_key`; CHECK lower-case and basic email format
password_hash	VARCHAR(100)	NN		CHECK must look like a bcrypt hash (`$2a/$2b/$2y$`, 60 chars), so plaintext is rejected
full_name	VARCHAR(120)	NN		CHECK not blank
role	VARCHAR(20)	NN		CHECK IN (`admin`, `recruiter`, `candidate`)
is_active	BOOLEAN	NN	`TRUE`	Deactivate instead of delete
created_at	TIMESTAMPTZ	NN	`now()`	
last_login_at	TIMESTAMPTZ			Added by ALTER (009); set by `POST /api/auth/login`. Ignored by the updated_at trigger (014)
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

Extra: `UK (user_id, role)` is the target of the profile tables' composite FKs. Index `users_role_idx`.

2. `companies`

Hiring organisations.

Column	Type	Null	Default	Constraints / notes
company_id	BIGINT	NN	ID	**PK**
name	VARCHAR(150)	NN		**UK**; CHECK not blank
industry	VARCHAR(120)			Widened from 80 by ALTER (009)
location	VARCHAR(120)			
website	VARCHAR(255)			CHECK starts with `http://` or `https://`
description	TEXT			

Column	Type	Null	Default	Constraints / notes
created_at	TIMESTAMPTZ	NN	`now()`	
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

3. `recruiters`

Profile for users with role `recruiter`. One recruiter works for one company.

Column	Type	Null	Default	Constraints / notes
user_id	BIGINT	NN		**PK**; **FK** `(user_id, role)` → users ON DELETE CASCADE
role	VARCHAR(20)	NN	`'recruiter'`	CHECK `= 'recruiter'` (makes the role FK work)
company_id	BIGINT	NN		**FK** → companies ON DELETE RESTRICT
designation	VARCHAR(80)			e.g. "Technical Recruiter"
phone	VARCHAR(20)			CHECK digits/spaces/dashes, optional leading `+`
created_at	TIMESTAMPTZ	NN	`now()`	
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

Extra: `UK (user_id, company_id)` is the target of `jobs_recruiter_fkey`.

4. `candidates`

Profile for users with role `candidate`.

Column	Type	Null	Default	Constraints / notes
user_id	BIGINT	NN		**PK**; **FK** `(user_id, role)` → users ON DELETE CASCADE
role	VARCHAR(20)	NN	`'candidate'`	CHECK `= 'candidate'`
phone	VARCHAR(20)			Same CHECK as recruiters
location	VARCHAR(120)			
headline	VARCHAR(150)			One-line professional summary
years_experience	NUMERIC(4,1)	NN	`0`	CHECK 0–60
highest_education	VARCHAR(120)			
created_at	TIMESTAMPTZ	NN	`now()`	
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

5. `skills`

Shared catalogue of skills used by candidates and jobs.

Column	Type	Null	Default	Constraints / notes
skill_id	BIGINT	NN	ID	**PK**
name	VARCHAR(60)	NN		Unique index on `lower(name)` (case-insensitive); CHECK not blank

Column	Type	Null	Default	Constraints / notes
category	VARCHAR(20)	NN		CHECK IN (`programming`, `frontend`, `backend`, `database`, `ml`, `data`, `devops`, `cloud`)

6. `candidate_skills` (M:N candidates ↔ skills)

Column	Type	Null	Default	Constraints / notes
candidate_id	BIGINT	NN		**PK** part; **FK** → candidates ON DELETE CASCADE
skill_id	BIGINT	NN		**PK** part; **FK** → skills ON DELETE RESTRICT
proficiency	SMALLINT	NN		CHECK 1 (beginner) – 5 (expert)
years_used	NUMERIC(4,1)	NN	`0`	CHECK 0–60

7. `jobs`

Job postings.

Column	Type	Null	Default	Constraints / notes
job_id	BIGINT	NN	ID	**PK**
company_id	BIGINT	NN		**FK** → companies ON DELETE RESTRICT
recruiter_id	BIGINT			**FK** `(recruiter_id, company_id)` → recruiters(user_id, company_id) ON DELETE SET NULL (recruiter_id)
title	VARCHAR(150)	NN		CHECK not blank
description	TEXT	NN		CHECK ≥ 50 characters (embedded for matching)
location	VARCHAR(120)	NN		
work_mode	VARCHAR(10)	NN	`'onsite'`	CHECK IN (`onsite`, `remote`, `hybrid`)
salary_min	NUMERIC(12,2)			CHECK ≥ 0
salary_max	NUMERIC(12,2)			CHECK `jobs_salary_range_check`: `salary_max ≥ salary_min`
currency	CHAR(3)	NN	`'INR'`	CHECK three upper-case letters (ISO 4217 style)
openings	SMALLINT	NN	`1`	CHECK 1–500
status	VARCHAR(10)	NN	`'open'`	CHECK IN (`draft`, `open`, `closed`)
applicant_count	INTEGER	NN	`0`	CHECK ≥ 0; denormalised count maintained by trigger `trg_applications_applicant_count` (011)
posted_at	TIMESTAMPTZ	NN	`now()`	
closes_at	DATE			CHECK `jobs_closes_after_posted_check` (added in 009)
employment_type	VARCHAR(20)	NN	`'full_time'`	Added in 009; CHECK IN (`full_time`, `part_time`, `contract`, `internship`)
min_experience_years	NUMERIC(4,1)	NN	`0`	Added in 009; CHECK 0–40

Column	Type	Null	Default	Constraints / notes
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at('applicant_count')`; changes to applicant_count alone do not count as an edit

Indexes: `jobs_company_id_idx`, `jobs_recruiter_id_idx`, **partial** `jobs_open_posted_at_idx (posted_at DESC) WHERE status = 'open'`.

8. `job_skills` (M:N jobs ↔ skills)

Column	Type	Null	Default	Constraints / notes
job_id	BIGINT	NN		**PK** part; **FK** → jobs ON DELETE CASCADE
skill_id	BIGINT	NN		**PK** part; **FK** → skills ON DELETE RESTRICT
is_required	BOOLEAN	NN	`TRUE`	FALSE = nice-to-have
min_proficiency	SMALLINT	NN	`3`	CHECK 1–5

9. `resumes`

Uploaded resume files and their extracted text.

Column	Type	Null	Default	Constraints / notes
resume_id	BIGINT	NN	ID	**PK**
candidate_id	BIGINT	NN		**FK** → candidates ON DELETE CASCADE
file_name	VARCHAR(255)	NN		Original file name
storage_path	VARCHAR(500)			**UK**; path under `uploads/`. NULL for seeded text-only resumes
mime_type	VARCHAR(100)	NN	`'application/pdf'`	CHECK IN (`application/pdf`, `text/plain`)
file_size_bytes	INTEGER			CHECK 1 B – 5 MB
extracted_text	TEXT	NN		CHECK ≥ 50 characters; this text is embedded
is_primary	BOOLEAN	NN	`FALSE`	**Partial UK** `resumes_one_primary_per_candidate`: one primary per candidate
uploaded_at	TIMESTAMPTZ	NN	`now()`	

10. `applications`

A candidate's application to a job. This is the centre of the hiring workflow.

Column	Type	Null	Default	Constraints / notes
application_id	BIGINT	NN	ID	**PK**
job_id	BIGINT	NN		**FK** → jobs ON DELETE RESTRICT; **UK** `(job_id, candidate_id)`
candidate_id	BIGINT	NN		**FK** → candidates ON DELETE CASCADE
resume_id	BIGINT			**FK** → resumes ON DELETE SET NULL
cover_letter	TEXT			

Column	Type	Null	Default	Constraints / notes
status	VARCHAR(15)	NN	`'applied'`	CHECK IN (`applied` , `shortlisted` , `interviewed` , `offered` , `rejected`). Transitions guarded by trigger (011). Since 018 a deferred constraint trigger checks at COMMIT that `interviewed` has an interview and `offered` has a pending offer
match_score	NUMERIC(5,4)			CHECK −1 to 1. **Real cosine similarity** `1 - (resume <=> job)` . NULL until embeddings exist, never faked
applied_at	TIMESTAMPTZ	NN	`now()`	
updated_at	TIMESTAMPTZ	NN	`now()`	CHECK `updated_at ≥ applied_at` ; set automatically by trigger since 010

Indexes: `applications_candidate_id_idx` , `applications_resume_id_idx` , `applications_job_status_idx (job_id, status)` .

11. `application_status_history`

Audit trail: one row per status change, written automatically by trigger `trg_applications_status_history` (011).

Column	Type	Null	Default	Constraints / notes
history_id	BIGINT	NN	ID	**PK**
application_id	BIGINT	NN		**FK** → applications ON DELETE CASCADE
old_status	VARCHAR(15)			NULL for the first entry; CHECK in the 5 statuses
new_status	VARCHAR(15)	NN		CHECK in the 5 statuses; CHECK `old_status IS DISTINCT FROM new_status`
changed_by	BIGINT			**FK** → users ON DELETE SET NULL
note	TEXT			
changed_at	TIMESTAMPTZ	NN	`now()`	

12. `interviews`

Column	Type	Null	Default	Constraints / notes
interview_id	BIGINT	NN	ID	**PK**
application_id	BIGINT	NN		**FK** → applications ON DELETE CASCADE; **UK** `(application_id, round_no)`
round_no	SMALLINT	NN		CHECK 1–10
scheduled_at	TIMESTAMPTZ	NN		Indexed. Must be in the future on INSERT and on reschedule (trigger `trg_interviews_guard` , 018)
duration_minutes	SMALLINT	NN	`45`	CHECK 15–480
mode	VARCHAR(10)	NN	`'video'`	CHECK IN (`video` , `phone` , `onsite`)
location_or_link	VARCHAR(500)			Meeting link or address. CHECK `interviews_location_required_check` (018): required unless mode = `phone`

Column	Type	Null	Default	Constraints / notes
interviewer_id	BIGINT			**FK** → recruiters ON DELETE SET NULL
status	VARCHAR(12)	NN	`'scheduled'`	CHECK IN (`scheduled` , `completed` , `cancelled` , `no_show`). New rows must be `scheduled` ; only `scheduled` rows can change; the other three are final and frozen (`trg_interviews_guard` , 018/019). `completed` / `no_show` only once `scheduled_at` has passed (021)
rating	SMALLINT			CHECK 1–5; CHECK only allowed when status = `completed`
feedback	TEXT			
created_at	TIMESTAMPTZ	NN	`now()`	
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

Also enforced by `trg_interviews_guard` (018): an interview can only be inserted for a `shortlisted` or `interviewed` application.

13. `offers`

Column	Type	Null	Default	Constraints / notes
offer_id	BIGINT	NN	ID	**PK**
application_id	BIGINT	NN		**FK** → applications ON DELETE CASCADE; **UK** (one offer per application)
salary_offered	NUMERIC(12,2)	NN		CHECK > 0
currency	CHAR(3)	NN	`'INR'`	CHECK three upper-case letters
joining_date	DATE	NN		Must be after today (India date, `local_today()`) on INSERT (`trg_offers_guard` , 018)
expires_on	DATE			CHECK `expires_on ≤ joining_date` . Not in the past on INSERT; an offer can be accepted/declined until the end of this day (`trg_offers_guard`)
status	VARCHAR(10)	NN	`'pending'`	CHECK IN (`pending` , `accepted` , `declined` , `withdrawn`). New rows must be `pending` ; only `pending` rows can change; the other three are final and frozen (`trg_offers_guard` , 018/019). Accept/decline also needs the application to be `offered` . Becomes `withdrawn` automatically when its application is rejected while the offer is still pending (`trg_applications_withdraw_offer` , 020). Once `accepted` , its application can no longer be rejected (`trg_applications_accepted_offer_guard` , 022)
offered_at	TIMESTAMPTZ	NN	`now()`	
responded_at	TIMESTAMPTZ			CHECK: NULL while pending, set once accepted/declined
updated_at	TIMESTAMPTZ	NN	`now()`	Added in 010; set automatically by trigger `set_updated_at()`

14. `notifications`

In-app messages shown to a user.

Column	Type	Null	Default	Constraints / notes
notification_id	BIGINT	NN	ID	**PK**
user_id	BIGINT	NN		**FK** → users ON DELETE CASCADE
type	VARCHAR(30)	NN		CHECK IN (`application_submitted`, `status_changed`, `interview_scheduled`, `offer_made`, `system`, and since 018 `interview_updated`, `offer_responded`, `offer_withdrawn`)
title	VARCHAR(150)	NN		CHECK not blank
message	TEXT	NN		
related_application_id	BIGINT			**FK** → applications ON DELETE SET NULL
is_read	BOOLEAN	NN	`FALSE`	**Partial** index `notifications_unread_idx WHERE NOT is_read`
created_at	TIMESTAMPTZ	NN	`now()`	Index `(user_id, created_at DESC)`

15. `resume_embeddings` (vector store)

Column	Type	Null	Default	Constraints / notes
resume_id	BIGINT	NN		**PK** and **FK** → resumes ON DELETE CASCADE (one vector per resume)
embedding	vector(384)	NN		CHECK unit length (`abs(vector_norm − 1) < 0.001`); **HNSW** index `vector_cosine_ops`
model_name	VARCHAR(100)	NN	`'sentence-transformers/all-MiniLM-L6-v2'`	Which model produced the vector
chunk_count	SMALLINT	NN		CHECK ≥ 1; number of text chunks averaged
source_hash	CHAR(64)	NN		CHECK 64 hex chars; SHA-256 of the embedded text (detects stale vectors)
created_at	TIMESTAMPTZ	NN	`now()`	

16. `job_embeddings` (vector store)

Same structure as `resume_embeddings`, with `job_id` as **PK** and **FK** → jobs ON DELETE CASCADE.

Bookkeeping (not an application table)

`tm_meta.schema_migrations(version PK, filename UK, checksum CHAR(64), applied_at)` is written only by `scripts/migrate.js`. It records which migration files have been applied and their SHA-256 checksums.